

Master of Data Science and Innovation



36118 Applied Natural Language Processing (ANLP)

Session 5

Dr. Antonette Shibani

Senior Lecturer, MDSI

Subject grades

For each coursework subject you complete you are awarded one of the following grades:

SUBJECT GRADES	MARK RANGE	DESCRIPTION
High Distinction (HD)	85-100%	Work of outstanding quality on all objectives of the subject.
Distinction (D)	75-84%	Work of superior quality on all objectives of the subject.
Credit (C)	65-74%	Work of good quality showing more than satisfactory achievement on all objectives of the subject.
Pass (P)	50-64%	Work showing a satisfactory achievement on all objectives of the subject.
Fail (X)	50% or more	Unsatisfactory performance in a compulsory component of the subject. A Fail(X) grade is awarded on the basis that compulsory component is essential for meeting the objectives of the subject.
Fail (Z)	0-49%	Work showing an unsatisfactory achievement on one or more objectives of the subject.

<https://www.uts.edu.au/for-students/current-students/managing-your-course/attendance-and-study-load/results/grading-schema-coursework-subjects>



Menti check-in

So far...

- Foundations of NLP (core concepts)
- NLP for text analysis, Python packages
- Text representation (Count, Tf-idf vectorizers)
- Unsupervised approaches (Clustering, Topic modelling)
- Supervised machine learning (Text classification)
- Sentiment analysis
- Word embeddings
- DL and neural networks
- Perceptron, Feed forward NN, CNNs, RNNs, LSTM



Today's session

- Transformers
- BERT
- GPT
- Language modeling
- Introduction to Large Language Models (LLMs)
- LLM-based tools

The slide features a solid blue background. There are several thin, white vertical lines of varying heights and positions. Some are located near the top, some near the bottom, and one pair is near the top right. The text is centered in the upper half of the slide.

Attention mechanism, Transformers

Let's explore a machine translation (English to Spanish) example...

The English sentence below combines two proverbs and requires understanding of idiomatic expressions and context:

“The early bird catches the worm, but the second mouse gets the cheese”

When RNN processes the sentence sequentially, it generates the Spanish translation word by word:

“El pájaro temprano atrapa el gusano, pero el segundo ratón obtiene el queso”

- Word-for-word literal translation
- Intended message of the idiom may not be conveyed because of the cultural context
- Sounds unnatural and awkward (esp., the second part)



A more idiomatic Spanish version might be:

“A quien madruga, Dios le ayuda, pero la paciencia también tiene su recompense”

(God helps those who wake up early, but patience also has its reward)

While a direct translation preserves the literal meaning, it doesn't quite capture the essence or fluency of the target language.

We want translations to capture the entire context!

Another example: English to French

English: “The agreement on the **European economic area** was signed in August 1992”

the European economic area → **la Européen économique aire**

One-for-one translation is incorrect **X**

A better translation might be:

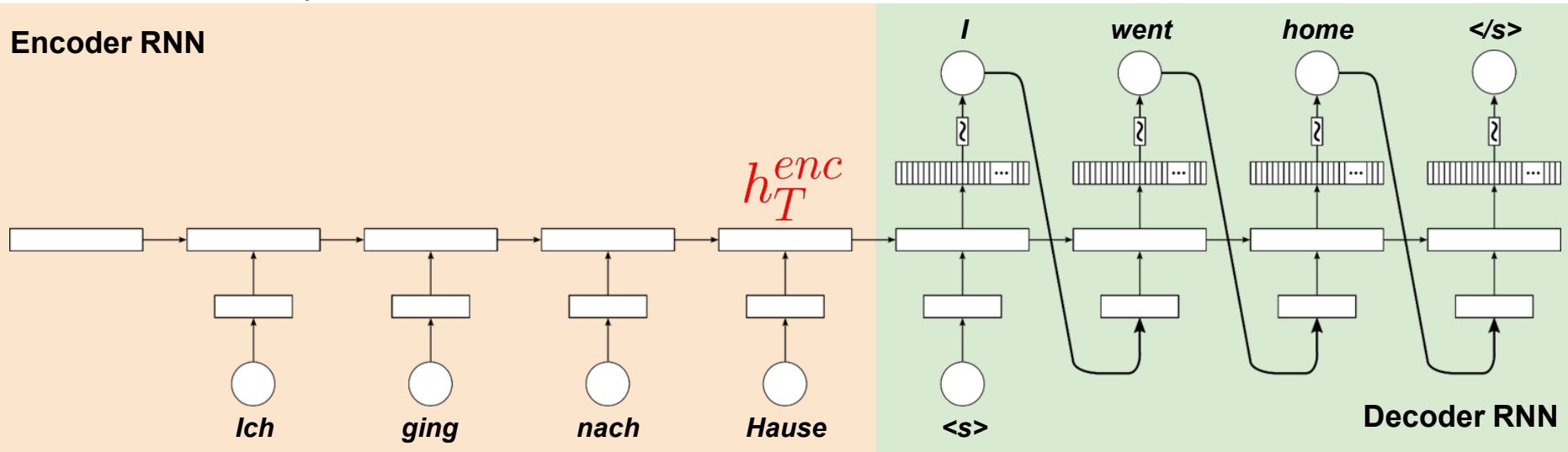
the European economic area → **la zone économique européenne**

Why? In French, European comes after economic (flipped),
‘Européenne’ word should be in the feminine form to match with la zone, and
while “aire” can mean “area” in some contexts, it's not the best choice here.

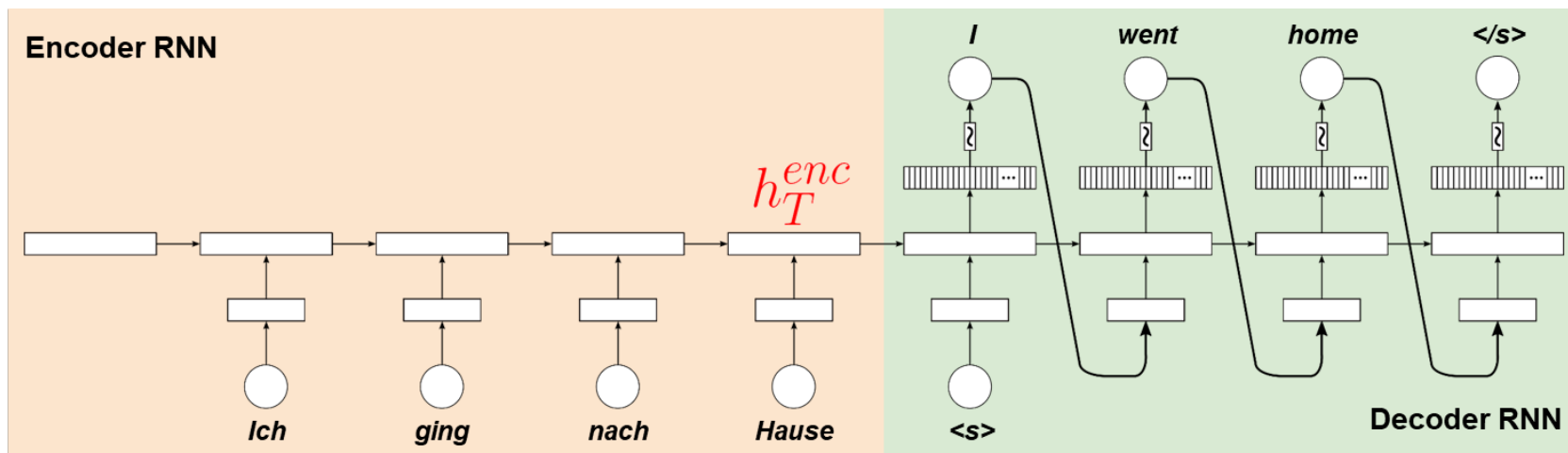
Language is complex and needs context (lots of it) for understanding!

RNN Machine Translation (covered in the last session)

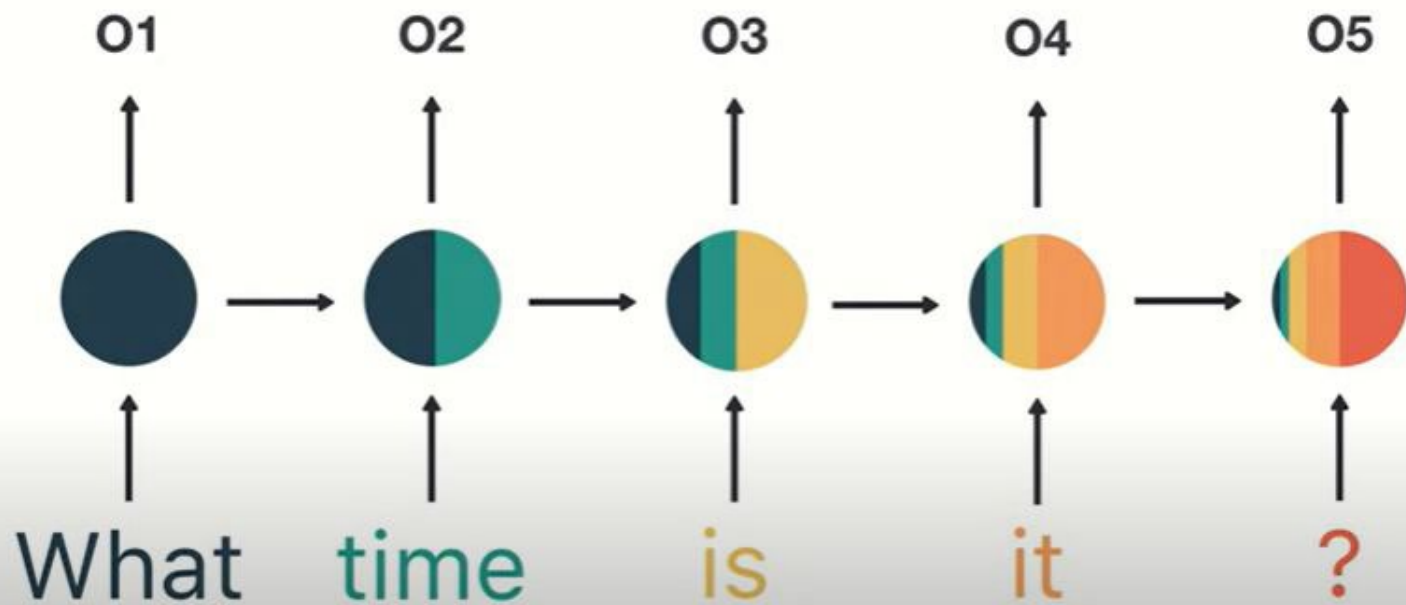
- Encoder-Decoder Architecture (Sutskever et al.; 2014)
 - Encoder: RNN to generate a latent representation (last hidden state h_T^{enc}) after processing the input sequence word by word, updating its hidden state along the way.
 - Decoder: RNN initialized with h_T^{enc} to generate output sequence. At each step, it produces a word and updates its hidden state.



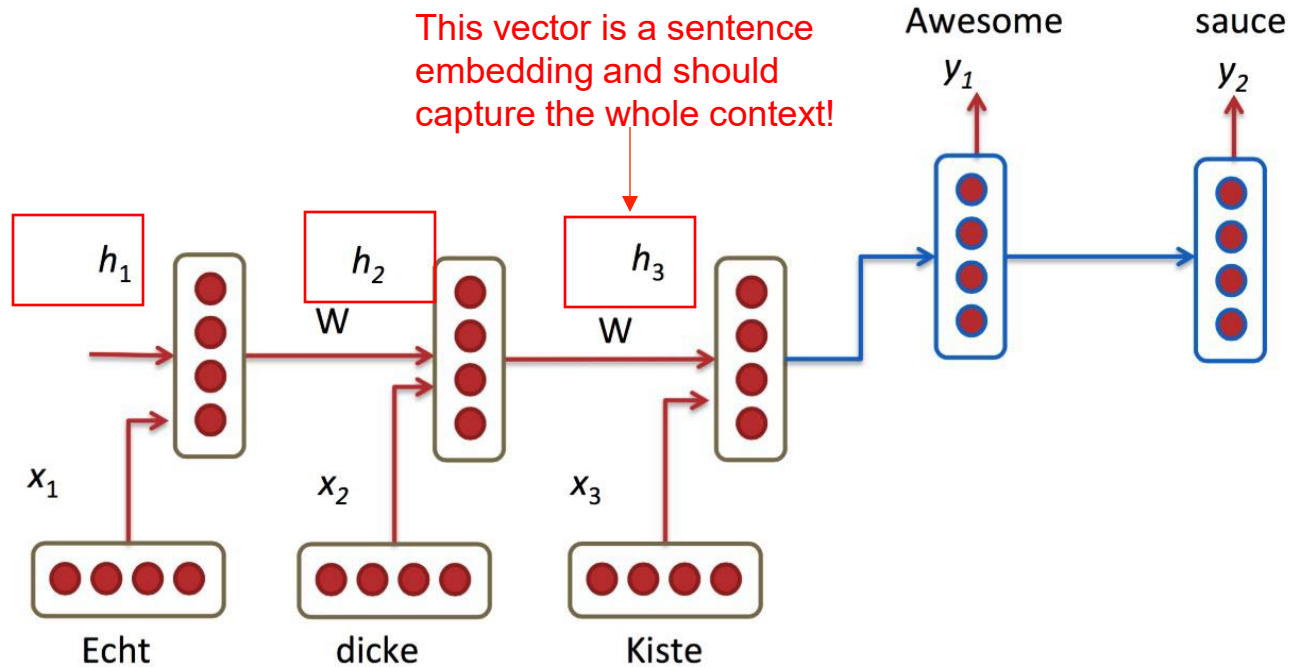
Quiz: What's the **decoder's** input for translating the given sentence?



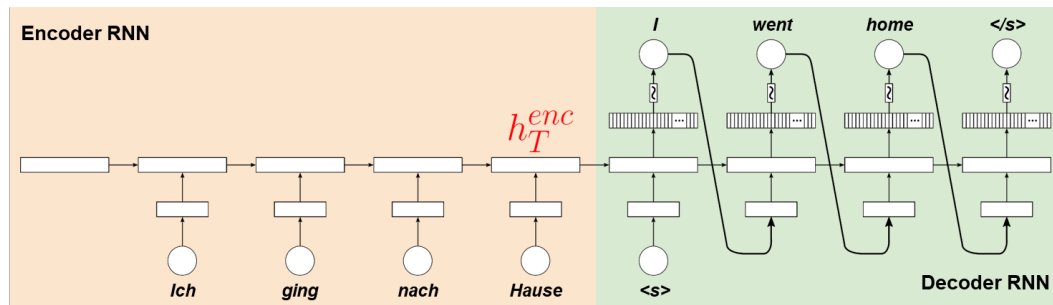
It's just the final hidden state!



Let's take a closer look at the encoder –decoder workings...



Assume you have a 50-word long sentence, what happens then?



The first word of the English translation is probably highly correlated with the first word of the source sentence (E.g. Ich -> I). But the decoder has to consider information from 50 steps ago, and that information needs to be somehow encoded in the vector.

In theory, LSTMs should be able to deal with long-range dependices, but in practice, they are still problematic!

RNN — Problem: (Very) Long Sequences

- Vanishing & Exploding Gradients problem
→ **Information dilutes over time** creating a bottleneck
(only recent information significantly influences the output, struggling to learn long-term dependencies)
- Processing is intrinsically sequential
→ **no parallelization**



Attention mechanism

Transformer architecture



Let's consider a complex, multi-part English paragraph and see how an RNN-based translation system compares with human translation:

English Paragraph: *“The ancient city, nestled in the valley, was a marvel of architecture. Its winding streets led to a grand plaza with a magnificent fountain. Despite its age, the city remained a bustling center of commerce.”*

The RNN processes the sentence word by word, updating its hidden state at each step:

1. "The" -> Initial state
2. "ancient" -> Update state
3. "city" -> Update state ... (continues for each word)

As the RNN progresses, it may start to lose information from the beginning of the sentence. By the time it reaches "Despite its age", it might have forgotten some details about the city's description.

RNN output: “La antigua ciudad, ubicada en el valle, era una maravilla arquitectónica. Sus calles sinuosas llevaban a una plaza con una fuente. A pesar de su edad, la ciudad era un centro de comercio.”

Back-translation: “The ancient city, located in the valley, was an architectural wonder. Its winding streets led to a square with a fountain. Despite its age, the city was a center of commerce.”

Original English text: "The ancient city, nestled in the valley, was a marvel of architecture. Its winding streets led to a grand plaza with a magnificent fountain. Despite its age, the city remained a bustling center of commerce."

Missing nuances and details

A human translator would instead:

- Read and understand the entire passage
- Consider the context, tone, and nuances
- Choose appropriate words and phrases in the target language
- Ensure consistency and coherence across the entire text

Human translation: *“La antigua ciudad, enclavada en el valle, era una maravilla de la arquitectura. Sus sinuosas calles conducían a una grandiosa plaza presidida por una magnífica fuente. A pesar de su antigüedad, la ciudad seguía siendo un bullicioso centro de comercio.”*

Back-translation: *“The ancient city, nestled in the valley, was a marvel of architecture. Its winding streets led to a grand plaza presided over by a magnificent fountain. Despite its antiquity, the city remained a bustling center of commerce.”*

We want to preserve most meaning and context like human translators!

Why transformers?

A transformer-based model would likely produce a translation more similar to the human translation: *“La antigua ciudad, enclavada en el valle, era una maravilla arquitectónica. Sus sinuosas calles conducían a una grandiosa plaza con una magnífica fuente. A pesar de su antigüedad, la ciudad seguía siendo un bullicioso centro comercial.”*

Key improvements:

- Maintains the "nestled" (enclavada) nuance
- Preserves the "grand" (grandiosa) and "magnificent" (magnífica) descriptors
- Captures the ongoing nature of the city's vitality with "seguía siendo" (remained)
- Retains the "bustling" (bullicioso) aspect of the commercial center

These improvements are possible because ***the transformer can attend to all parts of the input simultaneously, maintaining context and nuance*** throughout the translation process!

Our English to French example through Transformers

English: “The agreement on the **European economic area** was signed in August 1992”
the European economic area → **la Européen économique aire**

A better translation might be:

the European economic area → **la zone économique européenne**

Why? In French, European comes after economic (flipped),
‘Européenne’ word should be in the feminine form to match with la zone, and
while "aire" can mean "area" in some contexts, it's not the best choice here.



How do Transformers achieve this?

1. Positional Encoding
2. Attention
3. Self-attention

1. Positional Encoding

- The basic idea is to inject information about the position of each word in the sequence into the model.
- This is necessary because transformer models, unlike recurrent neural networks (RNNs), process all words in parallel and don't inherently have a sense of word order.

The cat sat

1 2 3

Word Vector + Positional Encoding = Position-Aware Word Representation

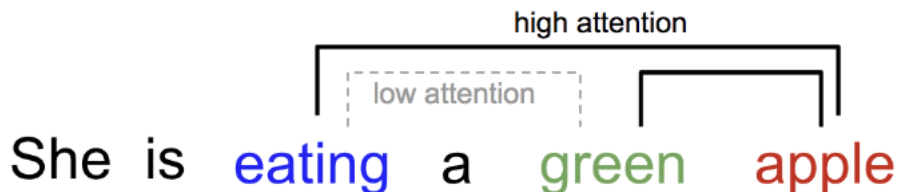
The	cat	sat ...	
[0.1, 0.2, 0.3, 0.4]	[0.5, 0.6, 0.7, 0.8]	...	Word Vector1, Word Vector 2,
[0.01, -0.01, 0.02, -0.02]	[0.02, -0.02, 0.04, -0.04]	...	Position 1, Position 2,
[0.11, 0.19, 0.32, 0.38]	[0.52, 0.58, 0.74, 0.76]	...	Combined Representation

2. Attention

<https://lilianweng.github.io/posts/2018-06-24-attention/>

Attention in deep learning can be broadly interpreted as a **vector of importance weights**:

To predict or infer one word in a sentence, we estimate using the attention vector **how strongly it is correlated with (or “attends to”) other words in the sequence**.



One word "attends" to other words in the same sentence differently – For example, when we see “eating”, we expect to encounter a food word very soon. The green color describes the food, but probably not so much with “eating” directly

Attention

- Attention is a mechanism that *allows the model to look at every single word in the original sentence when making a decision* about translation in the output sentence

→ la zone économique **européenne**

Model looks at European and economic when outputting the word **européenne**

- Allows the decoder to focus on different parts of the encoder's output when generating each word (Operates between the source (input) and the target (output) sequences)
- How does the model know which words to attend to?
→ Attention weights are learned from data

Which data,
and how much data?

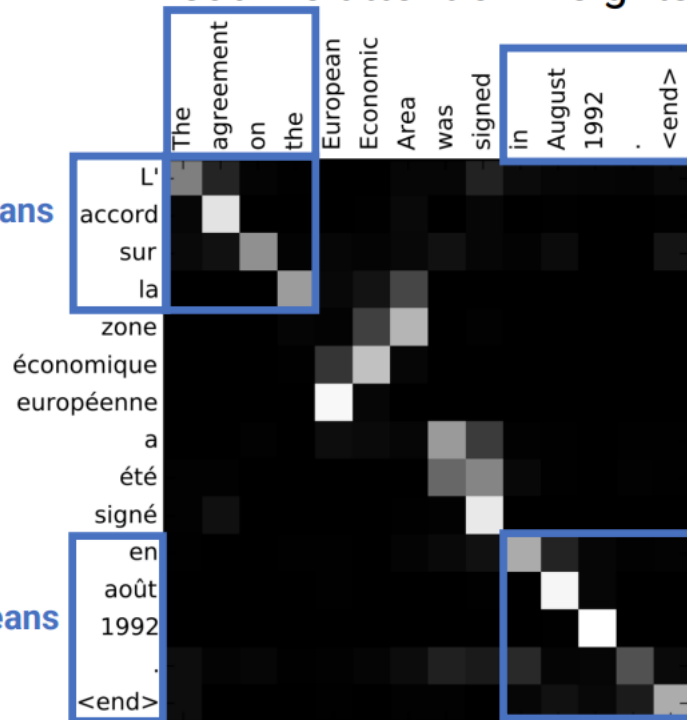
Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Diagonal attention means words correspond in order

Visualize attention weights $a_{t,i}$



Input: “The agreement on the European Economic Area was signed in August 1992.”

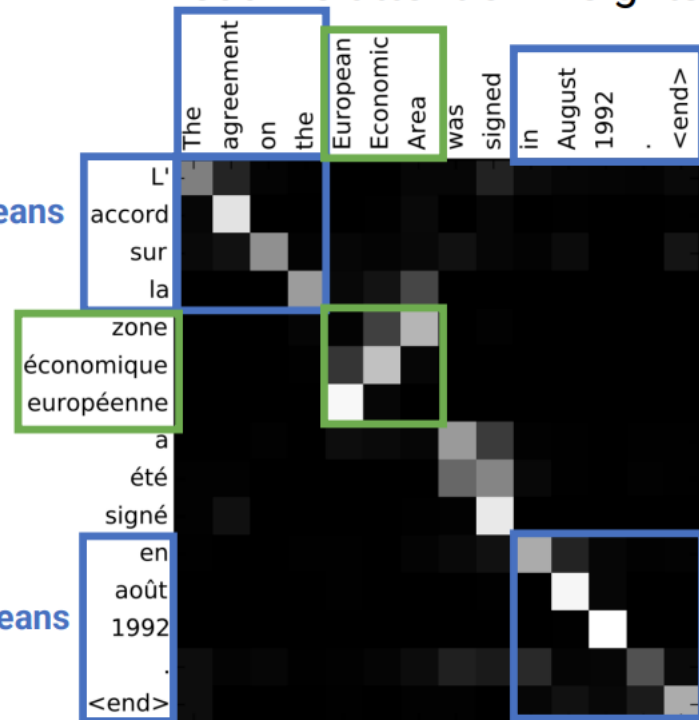
Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Attention figures out different word orders

Diagonal attention means words correspond in order

Visualize attention weights $a_{t,i}$



Attention Is All You Need

This is the original
'Transformer' paper

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

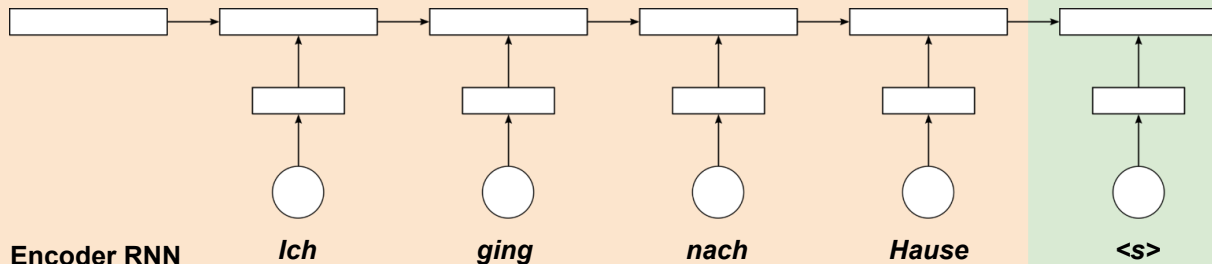
The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

Attention — Walkthrough

Attention Layer

Starting point

- Source sentence has been encoded using Encoder RNN (no changes here)
- First step of decoding process



Decoder RNN

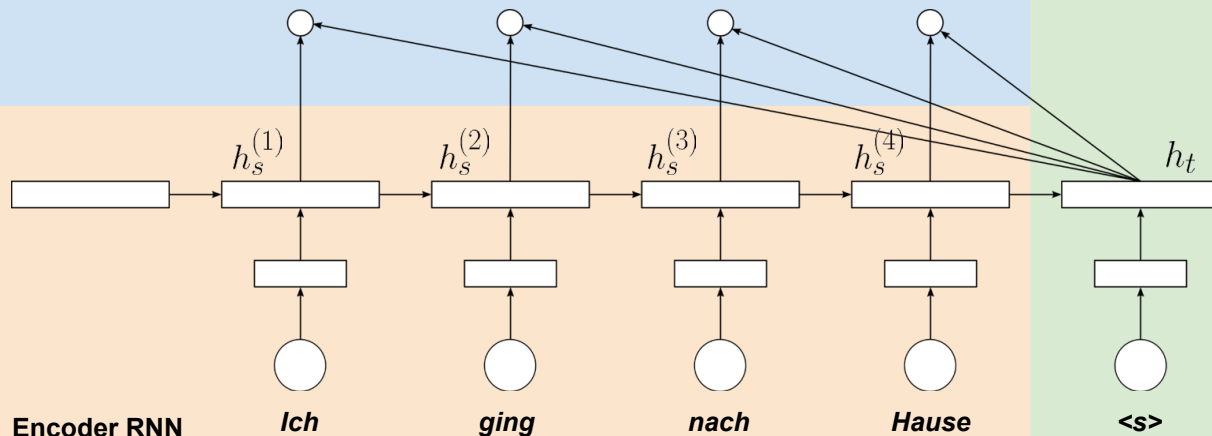
Attention — Walkthrough

Attention Layer

Step 1: Calculation of **Attention Scores**

- Attention scores = alignment between the current hidden state h_t of decoder and all hidden states of the encoder $h_s^{(i)}$
- Different scoring function applicable, e.g.:

$$e_i = \text{score}(h_t, h_s^{(i)}) = \begin{cases} h_t^T h_s^{(i)} & \text{dot product} \\ h_t^T \theta_a h_s^{(i)} & \text{general} \\ v_a^T \tanh(\theta_a [h_t, h_s^{(i)}]) & \text{concat} \end{cases}$$



Decoder RNN

Attention — Walkthrough

Attention Layer

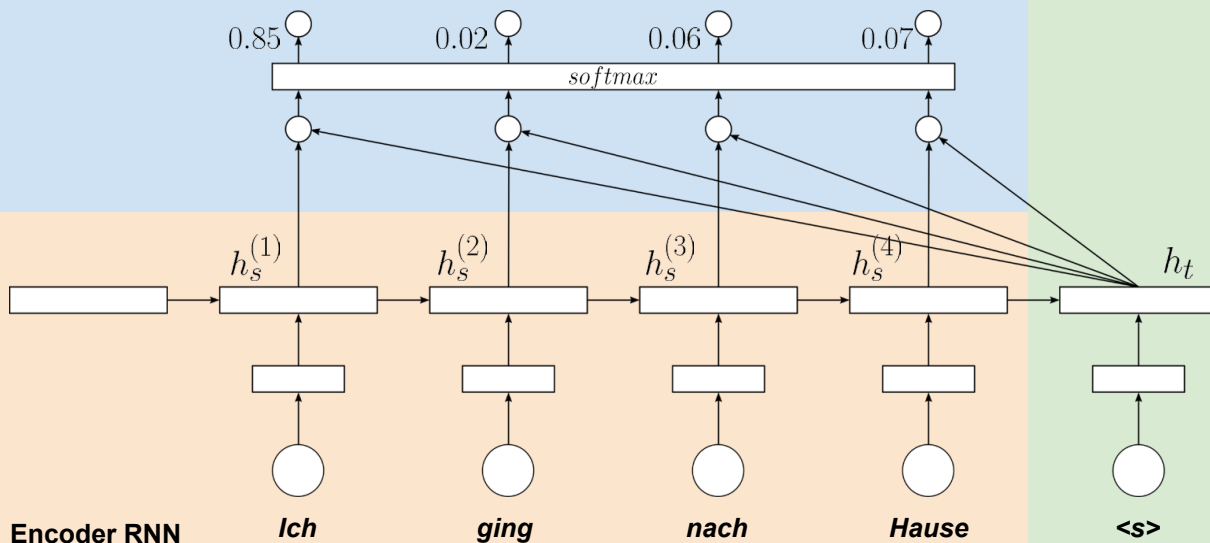
Step 2: Calculation of **Attention Weights**

- Attention weights a_i = attention scores pushed through a Softmax layer

$$a_i = \frac{\exp(e_i)}{\sum_i \exp(e_i)}$$

- Attention weights represent probabilities

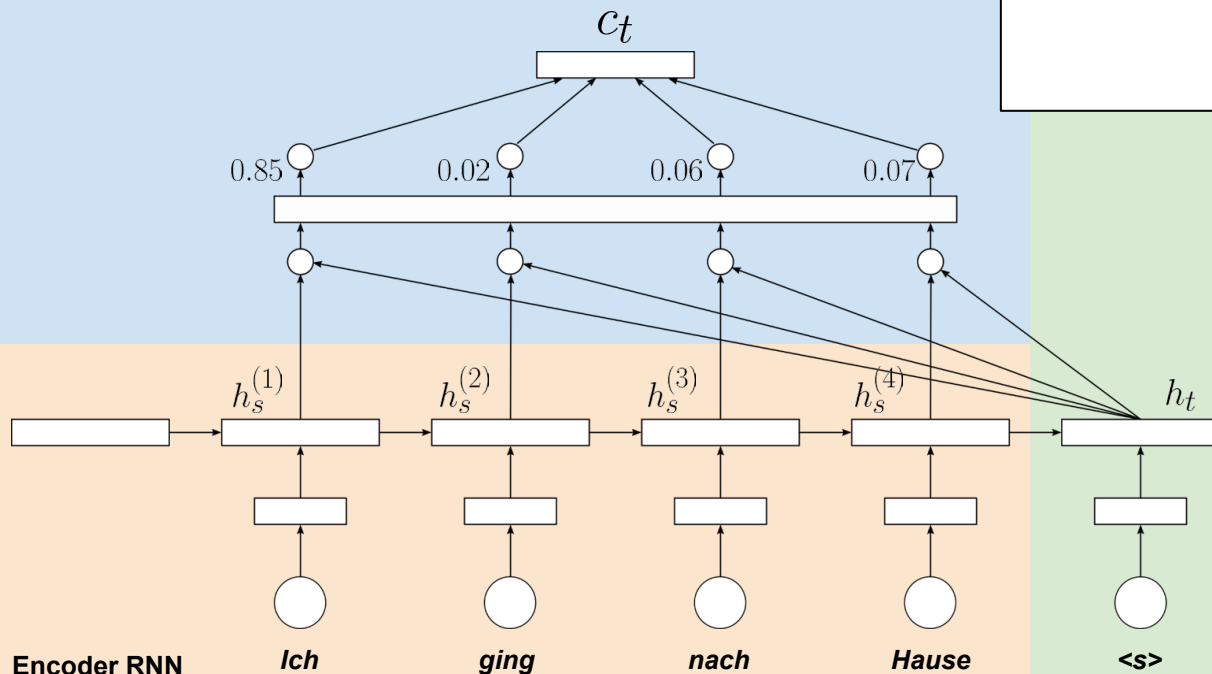
→ **Attention distribution**



Decoder RNN

Attention — Walkthrough

Attention Layer



Step 3: Calculation of **Context Vector**

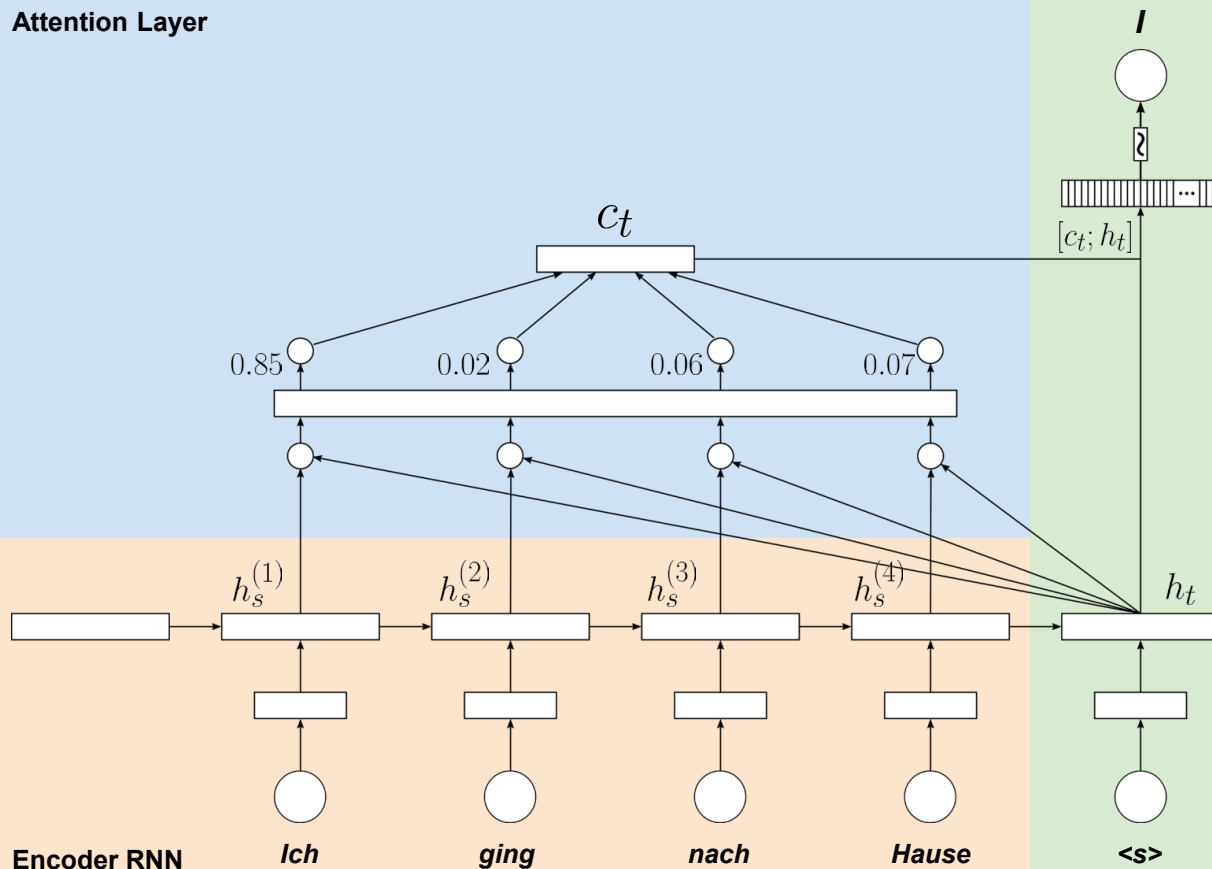
- Context vector c_t = weighted sum of all hidden states of the encoder $h_s^{(i)}$
- The weights are the attention weights

$$c_t = \sum_i a_i \cdot h_s^{(i)}$$

Decoder RNN

Attention — Walkthrough

Attention Layer



Step 4: Calculation of y_t

- Normal decoding step, BUT
- Use concatenation of c_t and h_t as input

$$y_t = \text{softmax}(\theta_{hy}[c_t, h_t])$$

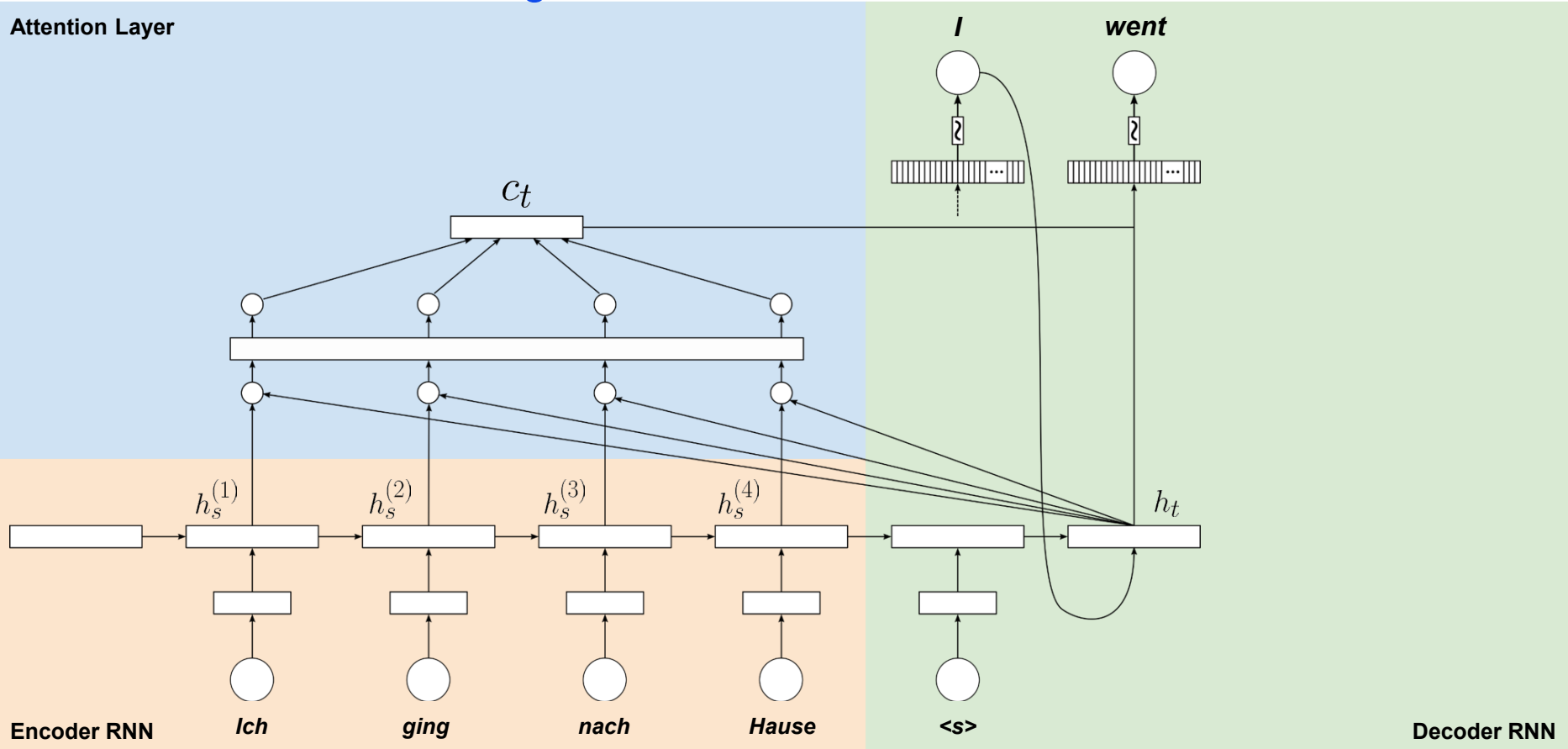
(most vanilla implementation)

Encoder RNN

Decoder RNN

Attention — Walkthrough

Attention Layer



Encoder RNN

Ich

ging

nach

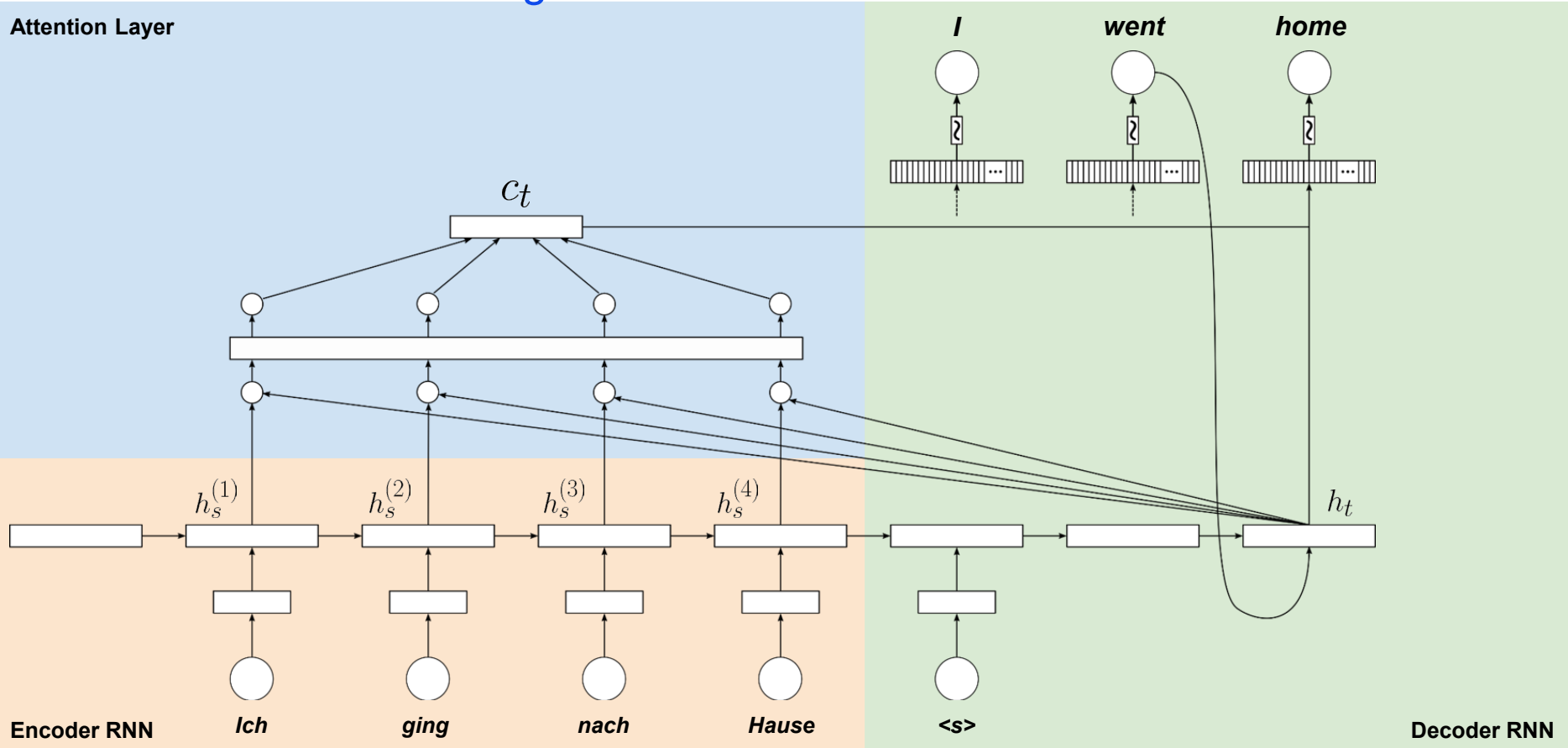
Hause

<s>

Decoder RNN

Attention — Walkthrough

Attention Layer



Encoder RNN

Ich

ging

nach

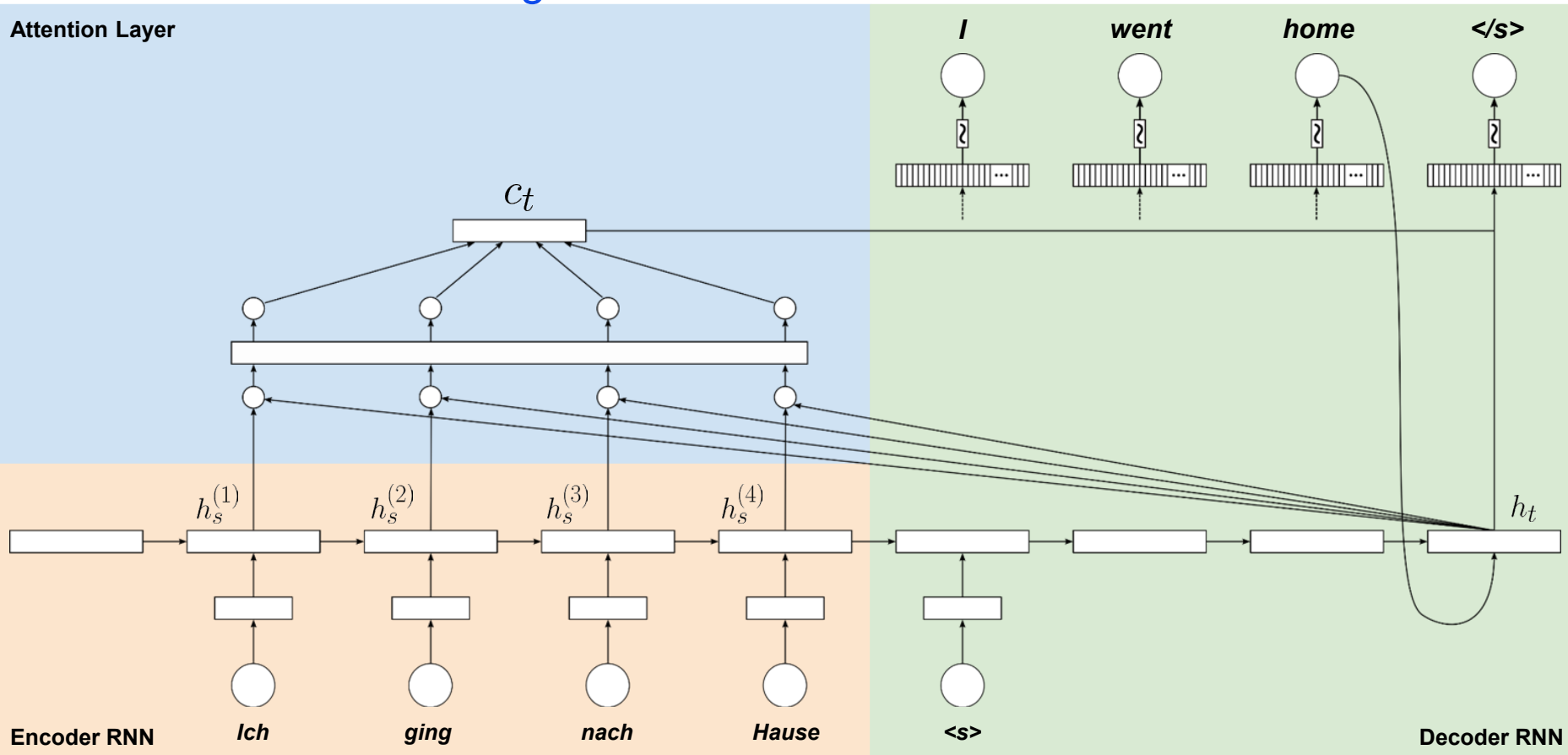
Hause

<s>

Decoder RNN

Attention — Walkthrough

Attention Layer



3. Self-attention

- Self-attention allows each position in a sequence (every word/ token) to attend to all positions in the *same* sequence regardless of how far apart the words are. This was the paper's 'innovation'!
- Creates rich, context-aware representations of words and phrases (Think of our vector representations), which can be adapted or fine-tuned for a wide range of NLP tasks. It helps disambiguate words, tense etc. [The better the internal model, the better it is at the language task!](#)

1. “**Server**, can I have the check?”
2. “Looks like I just crashed the **server**.”

For "server" in Sentence 1:

It would likely have high attention scores with "check", as this is a strong indicator of the restaurant context. It might also attend to "can I have", which is a typical request phrase used when addressing service staff.

For "server" in Sentence 2:

It would likely have high attention scores with "crashed", which is a term often associated with computer systems failing. It might also attend to "Looks like", which often precedes descriptions of technical issues.

How Self-Attention Works

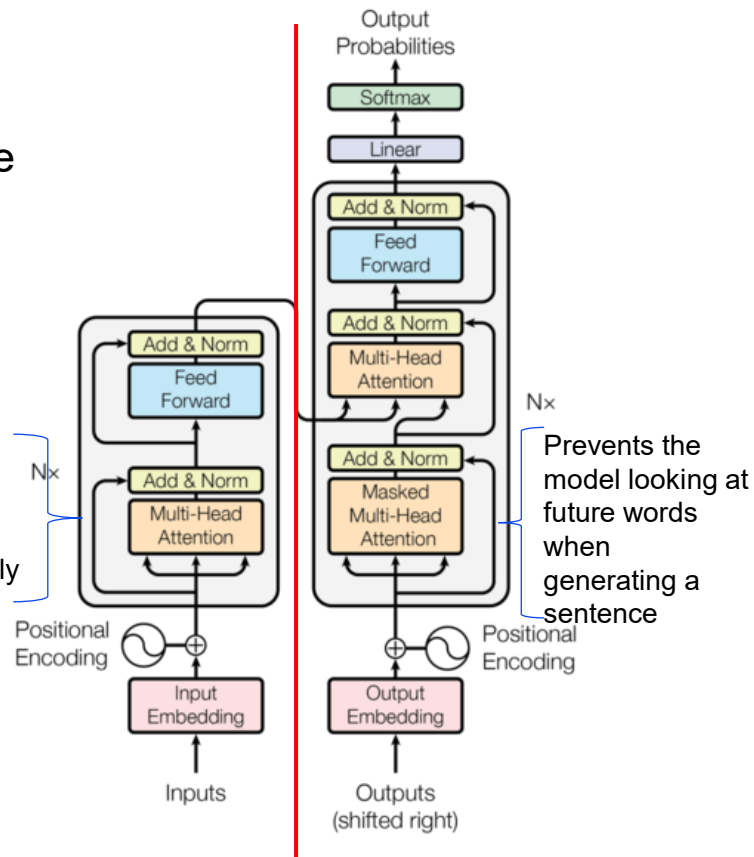
- For each word, create three vectors: Query (Q), Key (K), and Value (V)
- Calculate attention scores between the word and all other words (including itself)
- Use these scores to create a weighted sum of the values
- In practice, transformers use multi-head attention, repeating this process multiple times with different sets of weight matrices!

Transformer

We'll stop here, but you can go through each step of the transformer network training (and the underlying mathematical functions) here:

<https://jalammar.github.io/illustrated-transformer/>

Multiple self-attention mechanisms to look at different parts of the sentence simultaneously

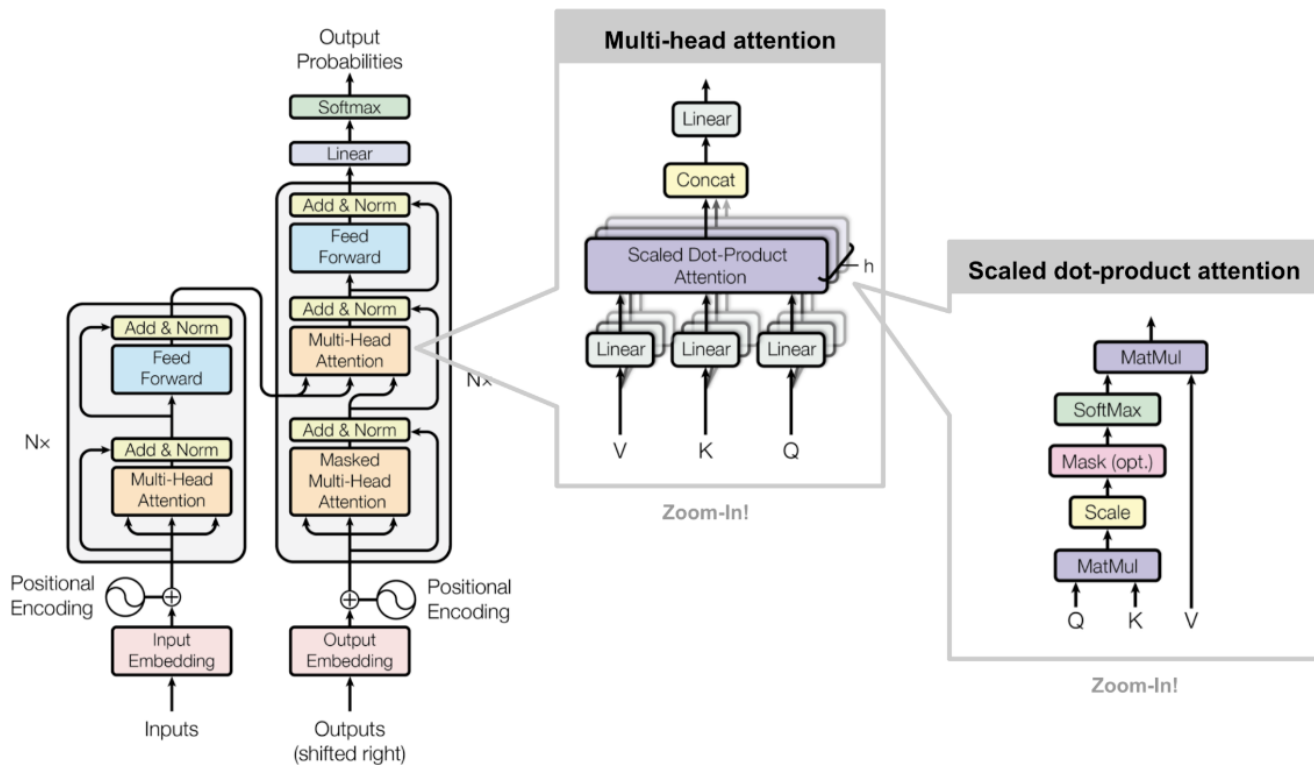


There is also an annotated transformer that runs through each step using Pytorch code (working notebook):

<https://nlp.seas.harvard.edu/annotated-transformer/>

Both parts have an understanding of language!

Full model architecture of Transformers



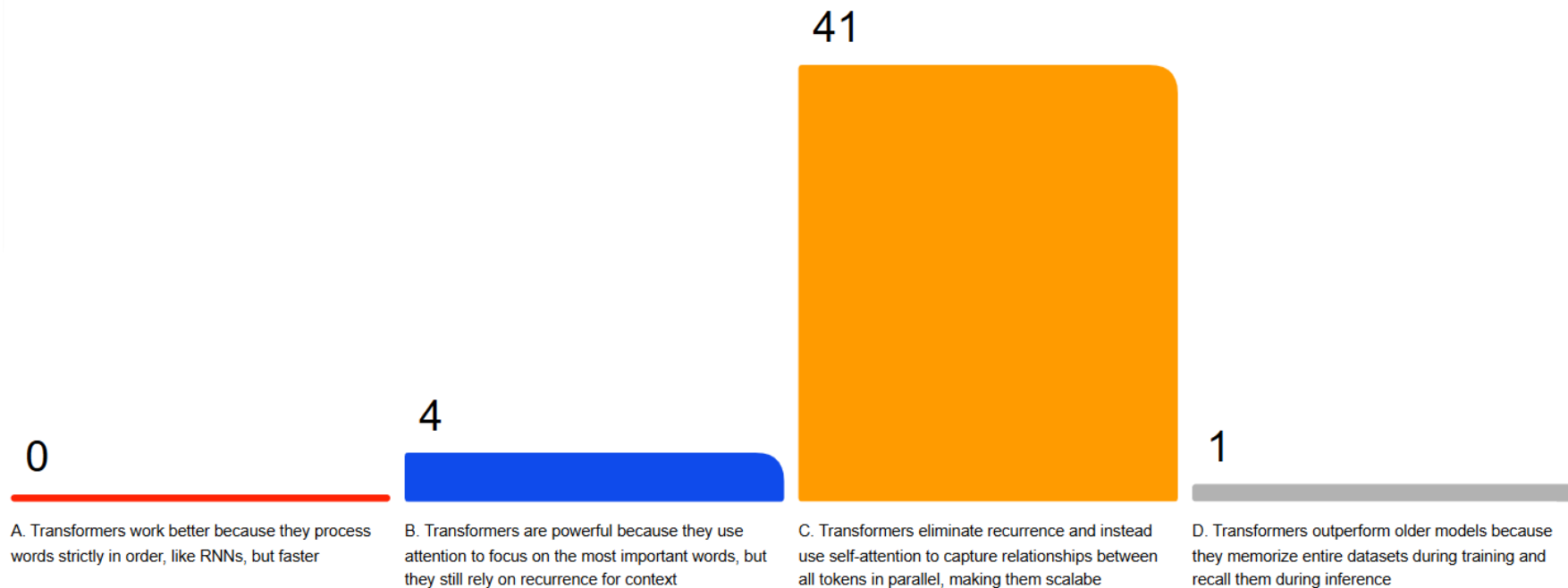
Feature	RNN	Transformer
Parallel Processing	Processes words sequentially, which is slow and can lead to loss of information	Processes all words in parallel, allowing for faster computation and better preservation of information
Attention Mechanism	Relies on a fixed-size hidden state to capture all information.	Uses self-attention to weigh the importance of all words in the sentence for each translation decision. This allows it to focus on relevant parts of the input for each output word.
Long-range Dependencies	Struggles with long-range dependencies due to the sequential nature of processing.	Can easily capture long-range dependencies thanks to the attention mechanism, which allows direct connections between any two words in the sentence
Positional Encoding	Inherently captures word order through sequential processing.	Uses positional encodings to inject information about word order , allowing it to understand sequence while processing in parallel
Multi-head Attention	Single perspective on the input sequence.	Uses multi-head attention , allowing it to capture different types of relationships between words (e.g., syntactic, semantic).

Join at menti.com | use code 5209 9171

Open Mentimote



Undoing Quiz - Why have Transformers have become the dominant architecture for NLP tasks?



Feedback on Transformers

- A. This is incorrect because Transformers do not process tokens strictly in sequence. Unlike RNNs, they use parallel processing with self-attention, which allows them to model long-range dependencies
- B. This is partially correct about attention but wrong about recurrence. Transformers completely remove recurrence. Context is captured through self-attention layers, which compute relationships bet
- C. Correct! The key innovation: parallelization and self-attention enable Transformers to handle long sequences efficiently and scale to large datasets.
- D. This is a common misunderstanding. While large models can memorize some patterns, their strength lies in learning generalizable representations through attention, not rote memorization. Overfitting is actually a problem to avoid, not a feature.

BERT

Bidirectional Encoder Representations from Transformers (BERT)

BERT is a revolutionary model that significantly improved how computers understand and work with human language.

- It is a language model, i.e. it represents the statistical relationships of the words in a language, Hence the part ‘**Representations**’
- BERT uses encoder only from the Transformer architecture
- BERT can be trained in an unsupervised way for representation learning (to understand language), and then we can fine-tune BERT on the downstream tasks for specific purposes (E.g. supervised classification)

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova
Google AI Language

{jacobdevlin, mingweichang, kentonl, kristout}@google.com

Abstract

We introduce a new language representation model called **BERT**, which stands for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute im-

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the same objective function during pre-training, where they use unidirectional language models to learn general language representations.

We argue that current techniques restrict the power of the pre-trained representations, especially for the fine-tuning approaches. The major limitation is that standard language models are unidirectional, and this limits the choice of architectures that can be used during pre-training. For example, in OpenAI GPT, the authors use a left-to-

BERT - advantages

BERT uses a "masked language model" (MLM) pre-training task, allowing it to **consider context from both directions** (bi-directional).

For example, in

"The man who entered the bank was carrying a fishing rod"

-> BERT can use both *"entered"* and *"fishing rod"* to understand the context of *"bank"* as a "riverbank"

BERT steps

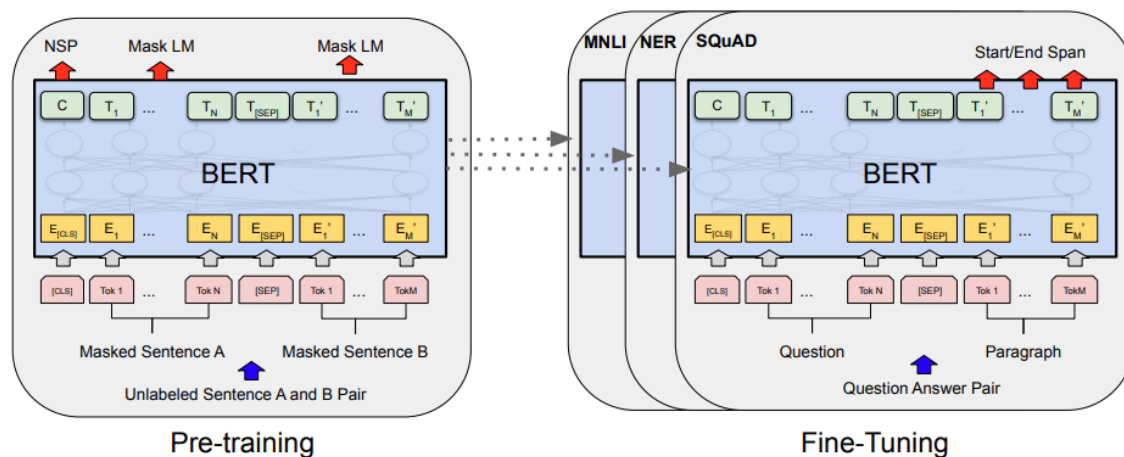
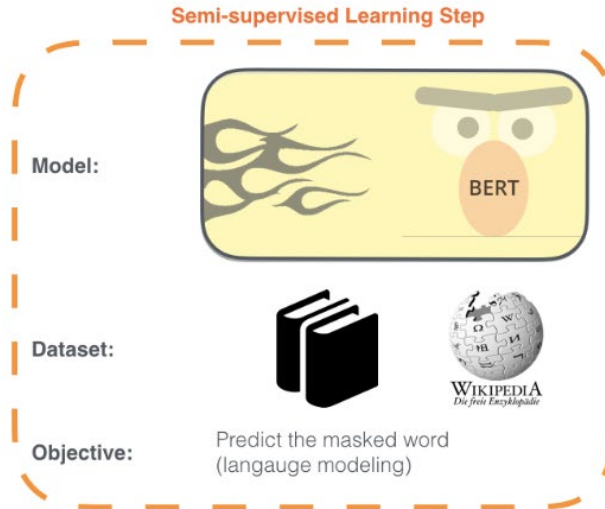


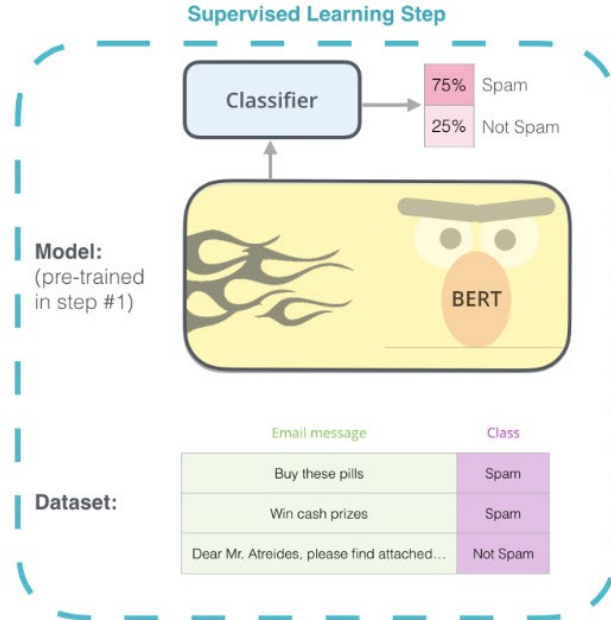
Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

1 - **Semi-supervised** training on large amounts of text (books, wikipedia..etc).

The model is trained on a certain task that enables it to grasp patterns in language. By the end of the training process, BERT has language-processing abilities capable of empowering many models we later need to build and train in a supervised way.



2 - **Supervised** training on a specific task with a labeled dataset.



The two steps of how BERT is developed. You can download the model pre-trained in step 1 (trained on un-annotated data), and only worry about fine-tuning it for step 2. [Source for book icon].

How BERT works

Step 1. Pre-training

a) Masked Language Model (MLM):

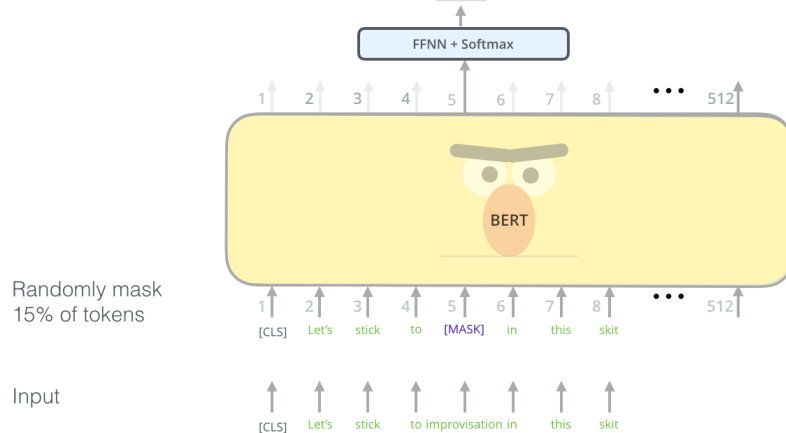
- 15% of input tokens are masked at random.
- The model is trained to predict these masked tokens.
- This allows bidirectional training while maintaining the benefits of language model pre-training.

b) Next Sentence Prediction (NSP):

- Binary classification task to predict if two sentences follow each other in the original text.
- Helps in learning sentence-level coherence.

Use the output of the masked word's position to predict the masked word

0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzyva



(We don't need pre-train if we are using an existing model – which is the usual case!)

Initial embedding

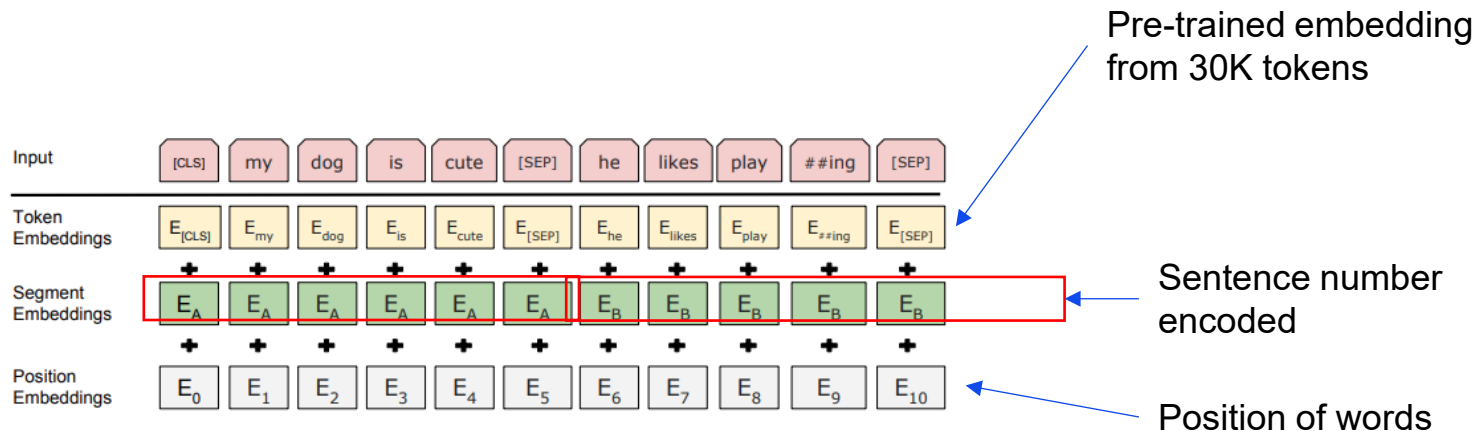


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

What's extra from the previous transformer architecture?

How BERT works

Step 2. Fine-tuning

- Task-specific fine-tuning involves minimal architecture changes.
- The entire pre-trained model is fine-tuned end-to-end on downstream tasks.

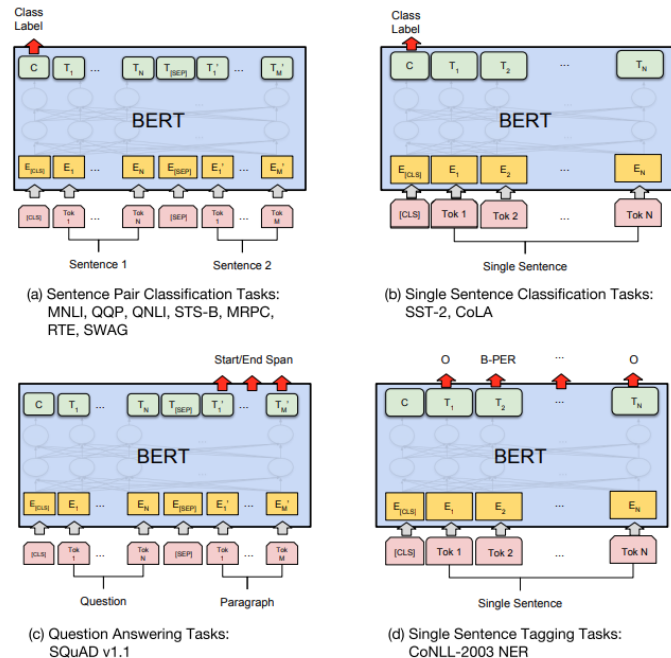


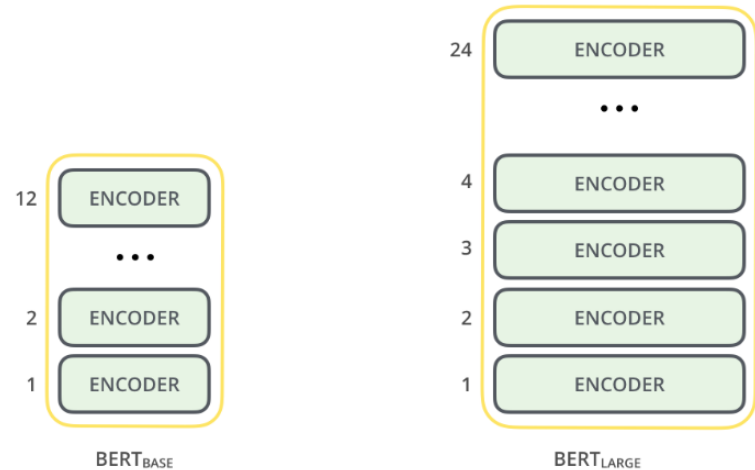
Figure 4: Illustrations of Fine-tuning BERT on Different Tasks.

BERT performs **transfer learning** by leveraging its pre-trained language understanding to quickly adapt to specific NLP tasks!

Two variants of BERT architecture

Based on *Encoders* in Transformer architecture.

- BERT Base: 12 layers (transformer blocks), 12 attention heads, and 110 million parameters
- BERT Large: 24 layers (transformer blocks), 16 attention heads and, 340 million parameters



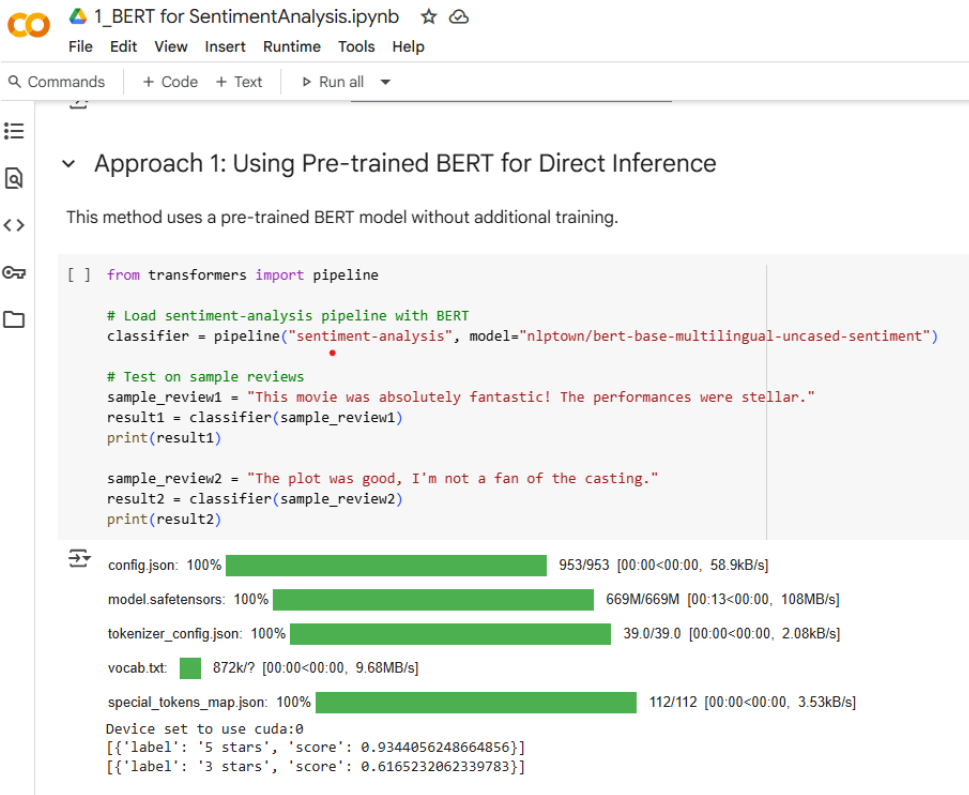
The BERT Base architecture has the same model size as OpenAI's (original) GPT.

<https://jalammar.github.io/illustrated-bert/>

How to apply BERT?

<https://colab.research.google.com/drive/1Cu1xGCjIPExZRTYqAVz9RU7rtvRf32-f?usp=sharing>

<https://tinyurl.com/ANLPsession5Part1>



The screenshot shows a Google Colab notebook interface. At the top, the title bar reads "1_BERT for SentimentAnalysis.ipynb" with icons for file, star, and share. Below the title bar is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A toolbar contains "Q Commands", "+ Code", "+ Text", and a "Run all" button with a dropdown arrow. The left sidebar shows a file explorer with a folder icon and a code editor icon. The main content area has a heading "Approach 1: Using Pre-trained BERT for Direct Inference" and a sub-heading "This method uses a pre-trained BERT model without additional training." Below this is a code cell with the following Python code:

```
[ ] from transformers import pipeline

# Load sentiment-analysis pipeline with BERT
classifier = pipeline("sentiment-analysis", model="nlpTown/bert-base-multilingual-uncased-sentiment")

# Test on sample reviews
sample_review1 = "This movie was absolutely fantastic! The performances were stellar."
result1 = classifier(sample_review1)
print(result1)

sample_review2 = "The plot was good, I'm not a fan of the casting."
result2 = classifier(sample_review2)
print(result2)
```

Below the code cell is a progress bar showing the status of various components:

- config.json: 100% [953/953] [00:00<00:00, 58.9kB/s]
- model.safetensors: 100% [669M/669M] [00:13<00:00, 108MB/s]
- tokenizer_config.json: 100% [39.0/39.0] [00:00<00:00, 2.08kB/s]
- vocab.txt: 872k/? [00:00<00:00, 9.68MB/s]
- special_tokens_map.json: 100% [112/112] [00:00<00:00, 3.53kB/s]

At the bottom, it says "Device set to use cuda:0" and displays the results of the sentiment analysis:

```
[{'label': '5 stars', 'score': 0.9344056248664856}]
[{'label': '3 stars', 'score': 0.6165232062339783}]
```

Tutorial

Fine-tuning BERT for sentiment analysis

<https://tinyurl.com/ANLPsession5Part1>

```
# Load the pre-trained tokenizer and model
# We'll use the base model (12 layers, 768 hidden units, 12 attention heads, 110M parameters), trained on lowercased text
tokenizer = BertTokenizerFast.from_pretrained("bert-base-uncased") # Using a pre-trained BERT tokenizer
model = BertForSequenceClassification.from_pretrained("bert-base-uncased")

tokenizer_config.json: 100% 48.04kB [00:00<00:00, 5.49kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 519kB/s]
tokenizer.json: 100% 400k/400k [00:00<00:00, 2.20MB/s]
config.json: 100% 670/670 [00:00<00:00, 11.1kB/s]
model.safetensors: 100% 440M/440M [00:11<00:00, 51.2MB/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

[] # Preprocess function which tokenizes, pads, and truncates the text sequences
def preprocess_function(examples):
    return tokenizer(examples["text"], padding=True, truncation=True)

[] tokenized_dataset = dataset.map(preprocess_function, batched=True) # Applies tokenization to the entire dataset
tokenized_dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"]) # Prepares the dataset for PyTorch and specifies which columns to include in the final dataset for training

Map: 100% 25000/25000 [00:28<00:00, 910.31 examples/s]
Map: 100% 25000/25000 [00:31<00:00, 871.32 examples/s]
Map: 100% 50000/50000 [00:55<00:00, 873.25 examples/s]

Defining a subset for training and test

[] # Let's take a subset of the data for faster training
train_dataset = tokenized_dataset["train"].shuffle(seed=42).select(range(2000))
test_dataset = tokenized_dataset["test"].shuffle(seed=42).select(range(1000))

[] # Training arguments
training_args = TrainingArguments(
    output_dir="results",
    eval_strategy="epoch",
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=10,
    logging_dir="logs",
    logging_steps=10,
    report_to="none" # Disables wandb and other integrations for logging
)

# Metrics
def compute_metrics(preds):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    acc = accuracy_score(labels, preds)
    return {"accuracy": acc}

# Trainer
trainer = GradientDescentTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)

# Train
trainer.train()

Epoch Training Loss Validation Loss Accuracy
1 0.359000 0.373567 0.880000
2 0.182000 0.482206 0.863000
3 0.041500 0.491233 0.897000

TrainOutput(global_step=750, training_loss=0.24662868338914487, metrics={'train_runtime': 672.0415, 'train_samples_per_second': 8.928, 'train_steps_per_second': 1.116, 'total_flos': 157866433218954487, 'train_loss': 0.24662868338914487, 'epoch': 3})

[] print(model) # Print model architecture summary

BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbedder(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
    )
  )
)
```

The slide features a solid blue background. There are seven short vertical white lines at the top, spaced out horizontally. Below them, the text 'Language Modelling' is centered in a white, sans-serif font. At the bottom, there are five longer vertical white lines, also spaced out horizontally.

Language Modelling

The cat sat on the mat

P(mat | The cat sat on
the)

The diagram illustrates the concept of a probability model for a word given its context. The word "the" is shown on the left, and the phrase "The cat sat on" is shown on the right, separated by a vertical bar "|". A blue bracket under "mat" in the original image (which is not present in the provided image) would indicate it is the next word. A blue bracket under "The cat sat on" indicates it is the context or prefix.

next word

context or prefix

Predict probability of the upcoming Word

$$\mathbf{P}(\underbrace{X_t}_{\text{next word}} \mid \underbrace{X_1, \dots, X_{t-1}}_{\text{context or prefix}})$$

Language Models (LMs)

- Predict the most appropriate next word based on the context of the given text
- E.g. “*Jenny dropped by the office for the keys so I gave them to [...]*,” a good model will determine that the missed word is likely to be a pronoun. Since the relevant piece of information here is **Jenny**, the most probable pronoun is ***she or her***
- What to note is that the model doesn’t focus on grammar, but rather on how words are used in a way that is similar to how people write
- Trained using a large dataset of text, LMs (often, LLMs) then use the patterns they learn from this training data to predict the next word or generate new text that is grammatically correct and semantically coherent

<https://www.youtube.com/watch?v=lnA9DMvHtfl#t=2m0s>

Why Care About Language Modeling?

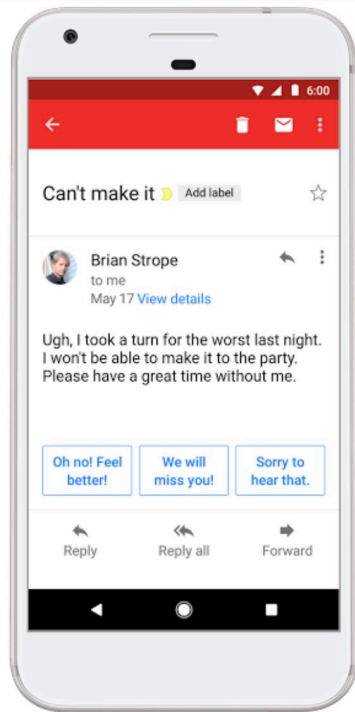
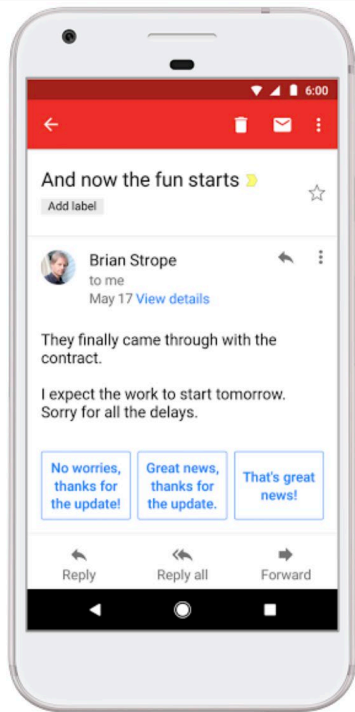
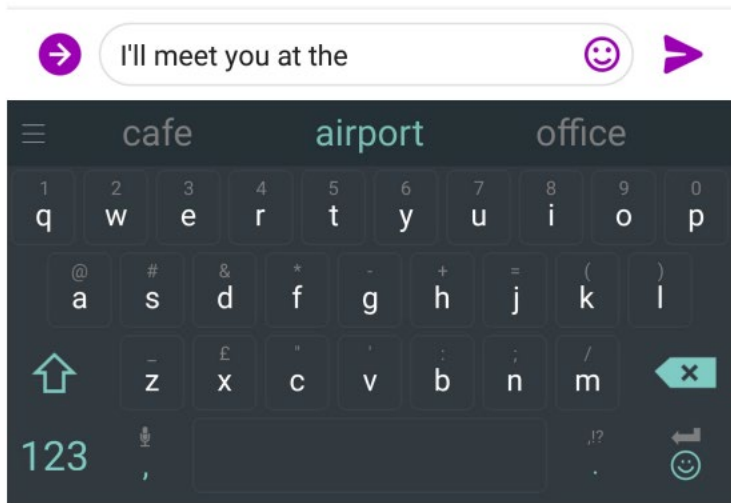
Language Modeling is a ~~subcomponent~~ **superset** of many tasks:

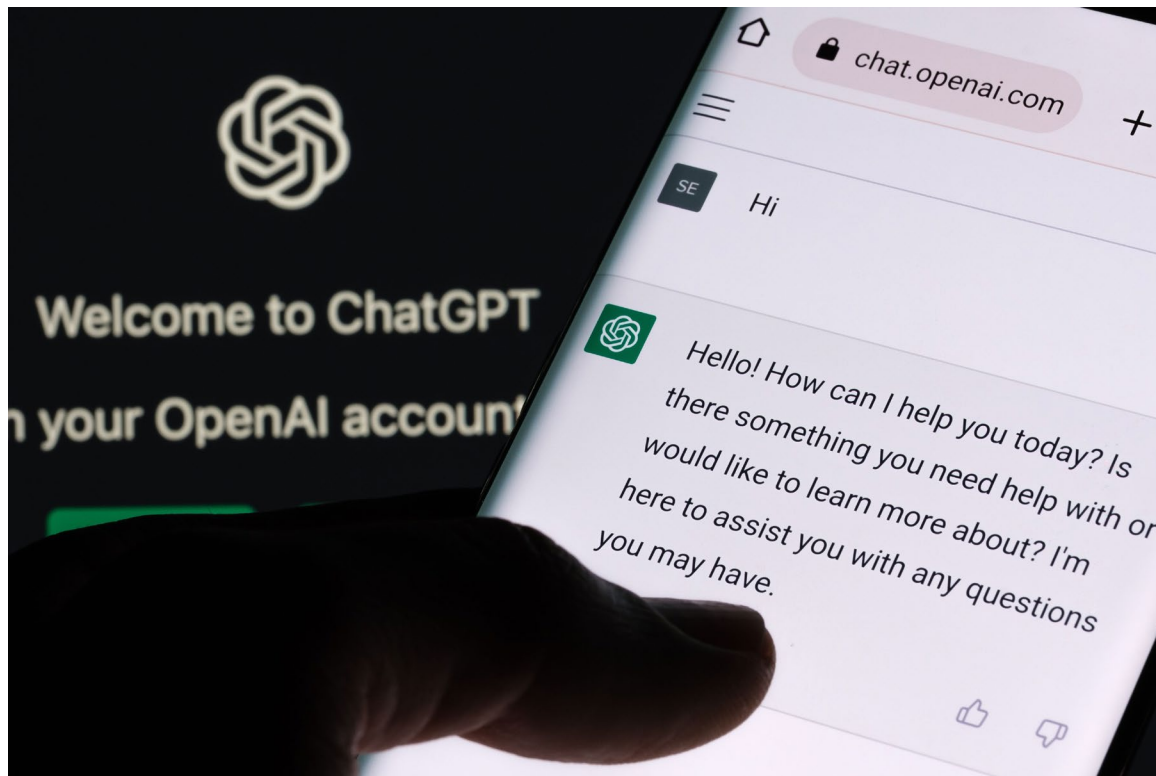
- Summarization
- Machine translation
- Spelling correction
- Dialogue etc.

Language Modeling is an effective proxy for **language understanding**.

- **Effective ability to predict forthcoming words** requires on **understanding of context/prefix**.

We use Language Models every day!





It can be misused too ...

This isn't a real
science article

A lot more about
harms later

Router: A Methodology for the Typical Unification of Access Points and Redundancy

Jeremy Stribling, Daniel Aguayo and Maxwell Krohn

ABSTRACT

Many physicists would agree that, had it not been for congestion control, the evaluation of web browsers might never have occurred. In fact, few hackers worldwide would disagree with the essential unification of voice-over-IP and public-private key pair. In order to solve this riddle, we confirm that SMPs can be made stochastic, cacheable, and interposable.

I. INTRODUCTION

Many scholars would agree that, had it not been for active networks, the simulation of Lamport clocks might never have occurred. The notion that end-users synchronize with the investigation of Markov models is rarely outdated. A theoretical grand challenge in theory is the important unification of virtual machines and real-time theory. To what extent can web browsers be constructed to achieve this purpose?

Certainly, the usual methods for the emulation of Smalltalk that paved the way for the investigation of rasterization do not apply in this area. In the opinions of many, despite the fact that conventional wisdom states that this grand challenge is continuously answered by the study of access points, we believe that a different solution is necessary. It should be noted that Router runs in $\Omega(\log \log n)$ time. Certainly, the shortcoming of this type of solution, however, is that compilers and superpages are mostly incompatible. Despite the fact that similar methodologies visualize XML, we surmount this issue without synthesizing distributed archetypes.

The rest of this paper is organized as follows. For starters, we motivate the need for fiber-optic cables. We place our work in context with the prior work in this area. To address this obstacle, we disprove that even though the much-touted autonomous algorithm for the construction of digital-to-analog converters by Jones [10] is NP-complete, object-oriented languages can be made signed, decentralized, and signed. Along these same lines, to accomplish this mission, we concentrate our efforts on showing that the famous ubiquitous algorithm for the exploration of robots by Sato et al. runs in $\Omega((n + \log n))$ time [22]. In the end, we conclude.

II. ARCHITECTURE

Our research is principled. Consider the early methodology by Martin and Smith; our model is similar, but will actually overcome this grand challenge. Despite the fact that such a claim at first glance seems unexpected, it is buffeted by previous work in the field. Any significant development of secure theory will clearly require that the acclaimed real-time algorithm for the refinement of write-ahead logging by Edward Feigenbaum et al. [15] is impossible; our application is no different. This may or may not actually hold in reality. We consider an application consisting of n access points. Next, the model for our heuristic consists of four independent components: simulated annealing, active networks, flexible modalities, and the study of reinforcement learning.

We consider an algorithm consisting of n semaphores. Any unproven synthesis of introspective methodologies will



Language models build probabilistic distribution over language.

An accurate distribution of language enables us to solve many important tasks that involve language communication.

The remaining question: how do we actually estimate this distribution?

Types of language models

1. Statistical language models use statistical patterns in the data to make predictions about the likelihood of specific sequences of words.

Based on a Markov assumption that the probability of a word combination (the future) depends only on the previous word (the present) and not the words that came before it (the past).

E.g. Calculating **n-gram** probabilities to generate possible text

2. Neural language models use [neural networks](#) to predict the likelihood of a sequence of words. These models are trained on a large corpus of text data and are capable of learning the underlying structure of the language

E.g. LLMs such as GPTs

N-gram Language Models

Terminology: *n*-gram is a chunk of *n* consecutive words:

E.g. “the cat sat on the mat”

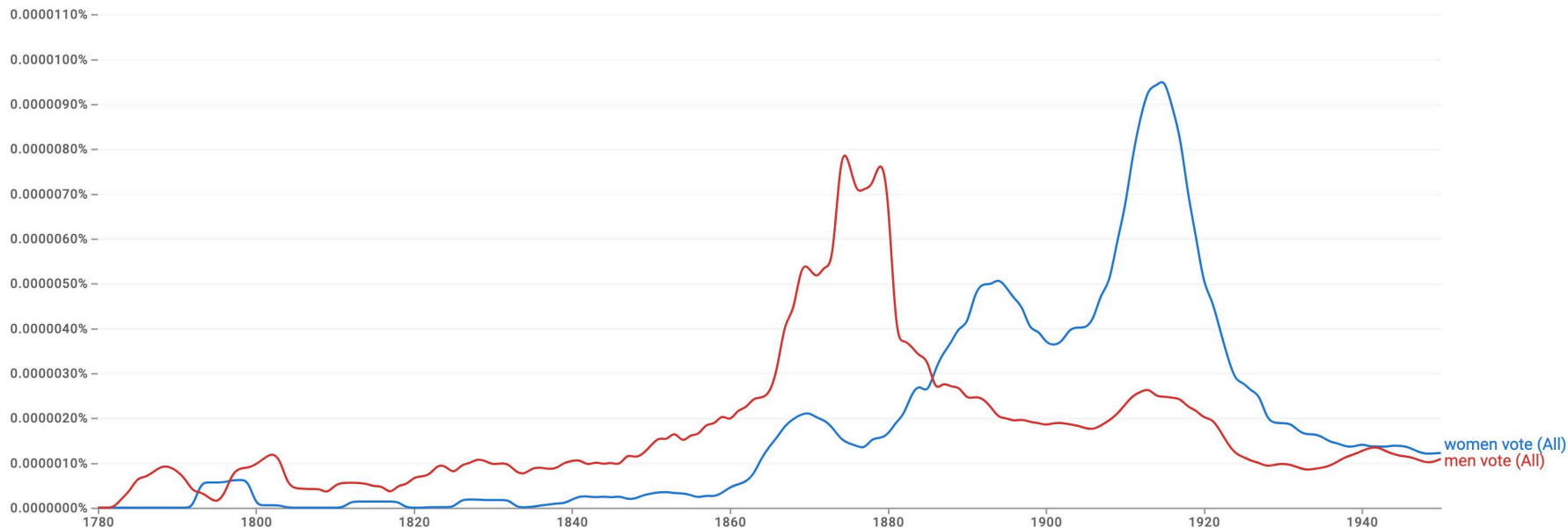
- **un**igrams: “cat”, “mat”, “sat”, ...
- **bi**grams: “the cat”, “cat sat”, “sat on”, ...
- **tri**grams: “the cat sat”, “cat sat on”, “sat on the”, ...
- **four**-grams: “the cat sat on”, “cat sat on the”, “sat on the mat”, ...

n-gram language model:

$$P(X_t | X_1, \dots, X_{t-1}) \approx P(X_t | \overbrace{X_{t-n+1}, \dots, X_{t-1}}^{n-1 \text{ elements}})$$

Pre-Computed N-Grams

Google Books Ngram Viewer



Google n-gram viewer <https://books.google.com/ngrams/>
Data: <http://storage.googleapis.com/books/ngrams/books/datasetv2.html>

Generation from N-Gram Models

You can build a simple **trigram** Language Model over a corpus in a few seconds on your laptop

-Try it out with a sample text!

The probabilities are estimated essentially by **counting word co-occurrences**.

N-Gram Models in Practice

If we sample from this model:

```
today the price of gold per ton , while production of shoe  
lasts and shoe industry , the bank intervened just after it  
considered and rejected an imf demand to rebuild depleted  
european stocks , sept 30 end primary 76 cts a share .
```

Surprisingly grammatical!

But **quite incoherent!** To improve coherence, one may consider increasing larger than 3-grams, but that would **worsen the sparsity problem!**

Sparsity problem in tri-gram language models

We often encounter sequences of words that haven't been seen in our training data leading to zero probabilities for these unseen sequences → **Require smoothing:**

- Assigning small, non-zero probabilities to unseen word sequences
- Adjusting the probabilities of seen sequences to make room for the unseen ones

Several **smoothing techniques** are used in language modeling:

- Add-one (Laplace) smoothing - adds 1 to all count frequencies
- Add-k smoothing: A generalization of add-one smoothing, where k is a small number (usually less than 1) added to all counts
- Many others....

N-Grams LMs and Long-range Dependencies

In general, count-based LMs are insufficient models of language because language has **long-distance dependencies**:

“**The computer** which I had just put into the machine room on the fifth floor **crashed**.”

We need better alternatives!

(and ways to assess the quality of LMs – covered later)

From Counting (N-Gram) to Neural Models

- n-gram models of text generation [Jelinek+ 1980's, ...]
 - Applications: Speech Recognition, Machine Translation
- “Shallow” statistical/neural language models (2000's) [Bengio+ 1999 & 2001, ...]

A Neural Probabilistic Language Model

Yoshua Bengio*, Réjean Ducharme and Pascal Vincent
Département d'Informatique et Recherche Opérationnelle
Centre de Recherche Mathématiques
Université de Montréal
Montréal, Québec, Canada, H3C 3J7
{bengioy, ducharme, vincentp}@iro.umontreal.ca



Attendance

The slide features a solid blue background. There are several thin, white vertical lines of varying heights and positions. Some are located near the top edge, while others extend from the bottom edge upwards. These lines appear to be decorative elements.

Large Language Models (LLMs) and GPTs

Large Language Models (LLMs)

Early one morning the sun was shining I was laying in bed
Wondering if she had changed at all if her hair was still ?

Transformers can help us build LLMs: <https://www.youtube.com/watch?v=YDiSFS-yHwk&t=2m26s>

(up to 4:48)

Generative Pre-trained Transformer (GPT)

Generative: This means the model can generate new, original text based on the input it receives.

Pre-trained: The model is initially trained on a vast amount of text data from the internet, books, and other sources. This gives it a broad understanding of language and general knowledge.

Transformer: We know how this specific NN architecture works – they are particularly good at processing sequential data like text.

GPT models are prime examples of **Large Language Models (LLMs)**. Advanced LLMs can ***perform new tasks with minimal examples or instructions***, a capability not present in smaller language models!

A similar structure to BERT

1. Generative pretraining

Train the GPT architecture to understand what language is

2. Fine tuning

Use transfer learning to apply GPT architecture on specific tasks

Problem: Fine-tuning requires a lot of data, overfitting is easy, not fluid for other processing!



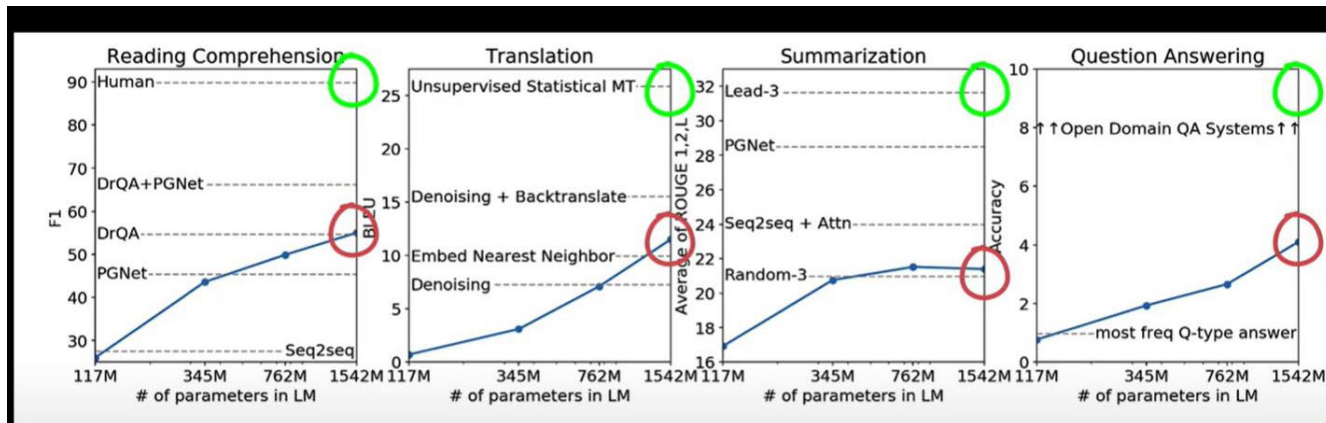
Meta-Learning

E.g. Zero-shot learning: Giving a prompt along with our input



Source: <https://www.youtube.com/watch?v=3lweGfgytgY>

GPT-2 and meta-learning did not perform that well though





What if the architecture is scaled even more?

L A R G E L A N G U A G E M O D E L S

- 1 WHAT IS AN LLM?
- 2 HOW DO THEY WORK?
- 3 BUSINESS APPLICATIONS

178 M. words → 1GB → 45TB → 1TB

LLM = DATA + ARCHITECTURE + TRAINING

IBM THE SKY IS

Subscribe

LLM FOUNDATION MODEL

this is a neural network and for GPT that is a transformer.

2:56 / 5:33

How Large Language Models Work:

<https://www.youtube.com/watch?v=5sLYAQS9sWQ>

GPT-3

- In 2020, GPT-3 took Silicon Valley by storm with its remarkable human-like text generation capabilities.
- It is a “*large language model*” (LLM) capable of producing fluent text after ingesting billions of words from books, articles, and websites. It was trained using 175 billion parameters!
- GPT-3 was so advanced that many individuals had difficulty distinguishing between news stories generated by the model and those written by human authors – Source paper: [“Language Models are Few-Shot Learners”](#) by OpenAI
- ChatGPT was created using GPT-3.5, specifically fine-tuned for conversational tasks. The publicly available version (November 30, 2022) made visible the possibilities to everyone.

- 
- Both GPT and BERT are examples of transformer architectures NLP in recent years
 - They have different strengths and weaknesses making them suitable for different types of tasks
 - GPT excels at generating long sequences of text with high accuracy whereas BERT focuses more on the understanding context within given texts in order to perform more sophisticated tasks such as question answering or sentiment analysis.

More advanced meta-learning in GPT-3

One-shot, few shot etc. – these are covered in the next session where we look at prompting techniques

One-shot Learning

- One example is passed through the “model context window”
- Reminder: No parameters in GPT-3 are updated.

Question => Answer

What is an apple? =>

Apple is an
amazing fruit.



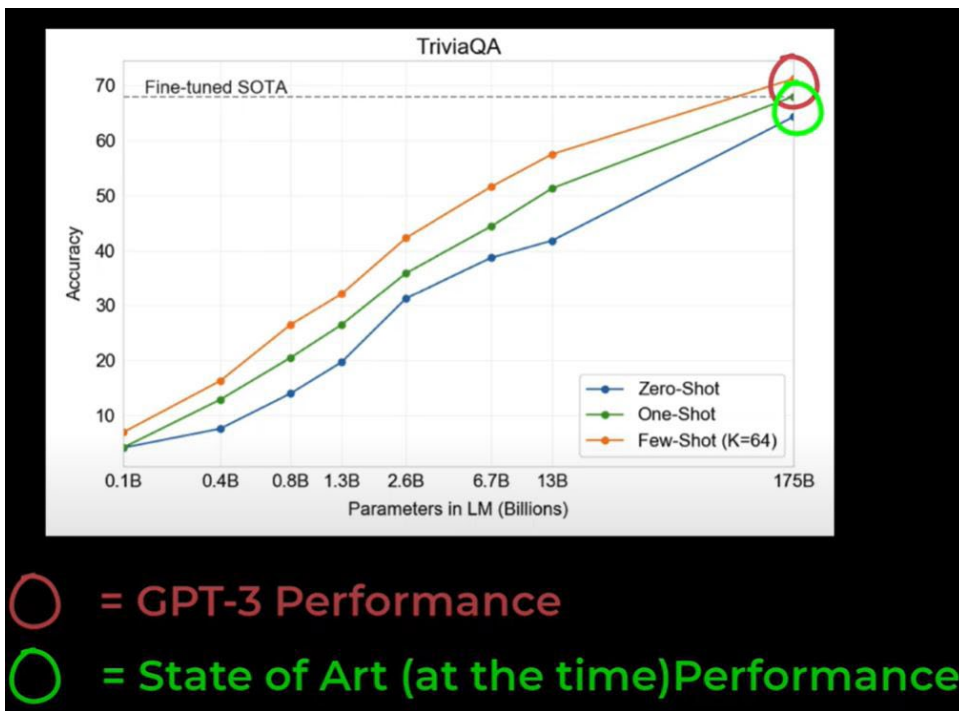
GPT-3



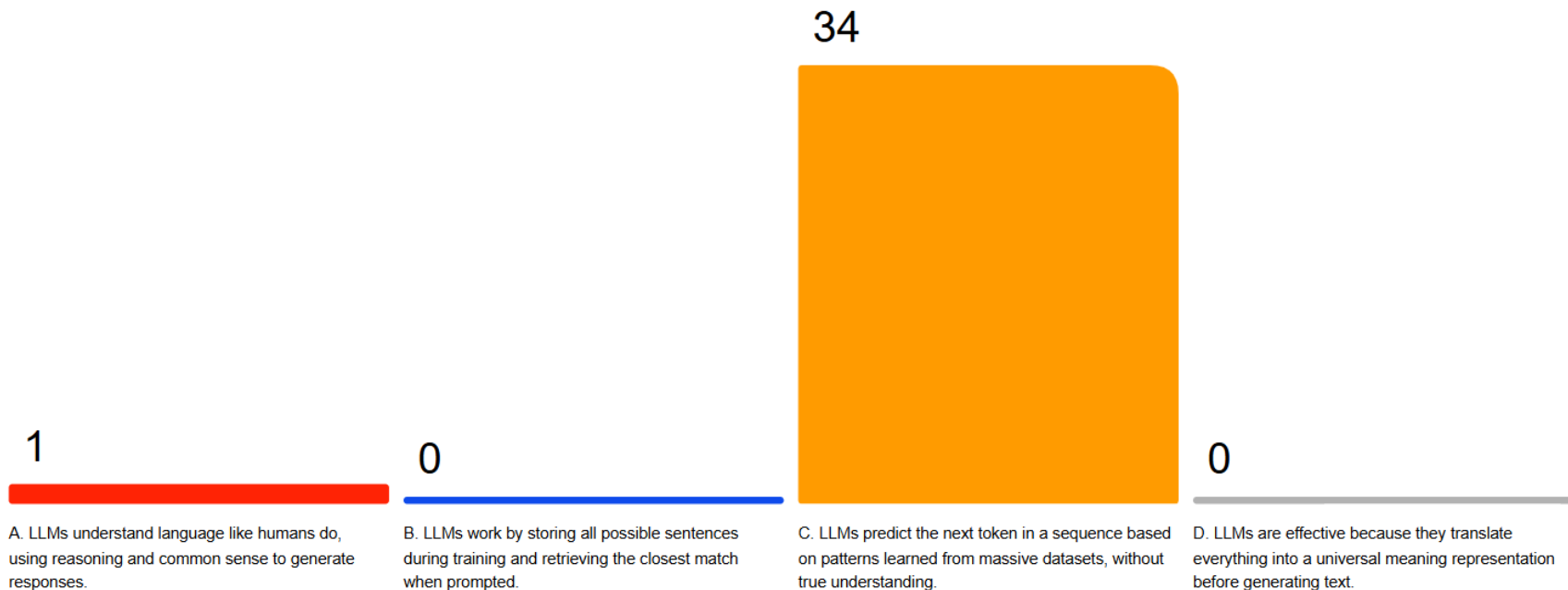
What is the
closest planet
to the sun?

Mercury is the
closest planet
to the sun

GPT-3 meta-learning has surpassed even fine-tuned models in certain tasks!



Undoing Quiz - Why are Large Language Models like GPT so effective at generating human-like text?



Feedback on LLMs

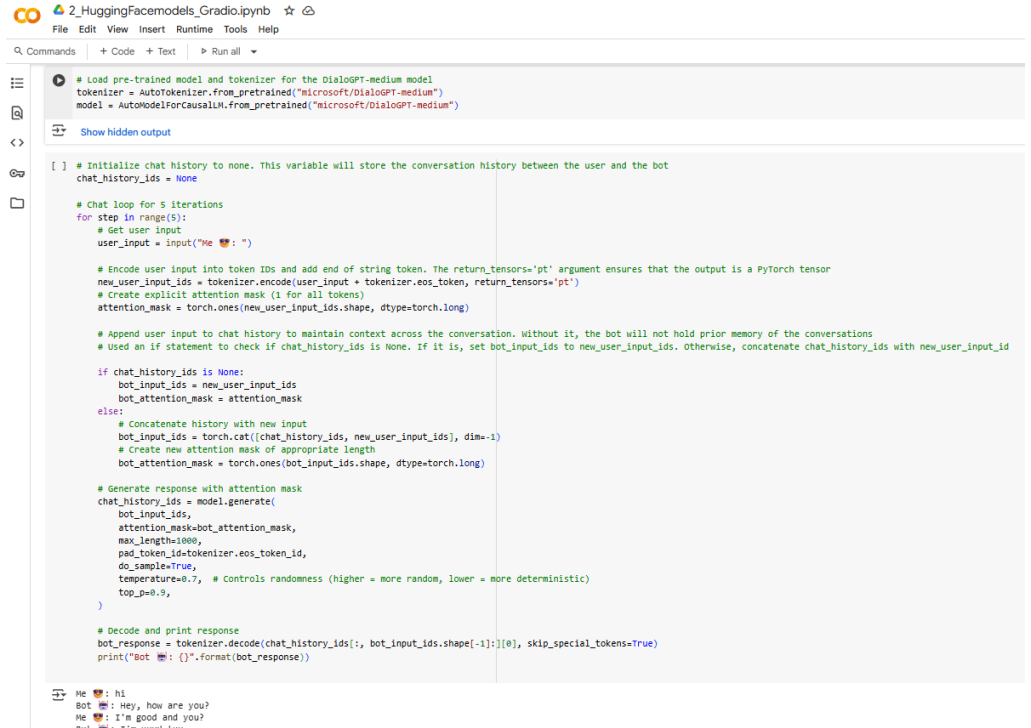
- A. This is incorrect because LLMs do not have human-like understanding or reasoning. They operate statistically, predicting the most likely next token based on learned patterns, not genuine comprehension.
- B. This is a common misunderstanding. LLMs do not memorize all sentences. Instead, they learn probabilistic relationships between tokens, enabling them to generate novel text rather than retrieve exact matches.
- C. Correct! LLMs are predictive models. They excel at pattern recognition and token prediction, which creates the illusion of understanding.
- D. This is incorrect. LLMs do not rely on a universal meaning representation. They work directly with token embeddings and attention mechanisms to model relationships, not semantic universals.

Tutorial

Let's apply a pre-defined GPT model to create a chatbot:

<https://colab.research.google.com/drive/1rori6P9yiu-n3-HOINSWV-IRUUDKv?usp=sharing>

<https://tinyurl.com/ANLPsession5Part2>



```
2_HuggingFaceModels_Gradio.ipynb
File Edit View Insert Runtime Tools Help

[ ] Commands + Code + Text + Run all

# Load pre-trained model and tokenizer for the DialoGPT-medium model
tokenizer = AutoTokenizer.from_pretrained("microsoft/DialoGPT-medium")
model = AutoModelForCausalLM.from_pretrained("microsoft/DialoGPT-medium")

Show hidden output

[ ] # Initialize chat history to none. This variable will store the conversation history between the user and the bot
chat_history_ids = None

# Chat loop for 5 iterations
for step in range(5):
    # Get user input
    user_input = input("Me 🗨️: ")

    # Encode user input into token IDs and add end of string token. The return_tensors='pt' argument ensures that the output is a PyTorch tensor
    new_user_input_ids = tokenizer.encode(user_input + tokenizer.eos_token, return_tensors='pt')
    # Create explicit attention mask (1 for all tokens)
    attention_mask = torch.ones(new_user_input_ids.shape, dtype=torch.long)

    # Append user input to chat history to maintain context across the conversation. Without it, the bot will not hold prior memory of the conversations
    # Used an if statement to check if chat_history_ids is None. If it is, set bot_input_ids to new_user_input_ids. Otherwise, concatenate chat_history_ids with new_user_input_id

    if chat_history_ids is None:
        bot_input_ids = new_user_input_ids
        bot_attention_mask = attention_mask
    else:
        # Concatenate history with new input
        bot_input_ids = torch.cat([chat_history_ids, new_user_input_ids], dim=-1)
        # Create new attention mask of appropriate length
        bot_attention_mask = torch.ones(bot_input_ids.shape, dtype=torch.long)

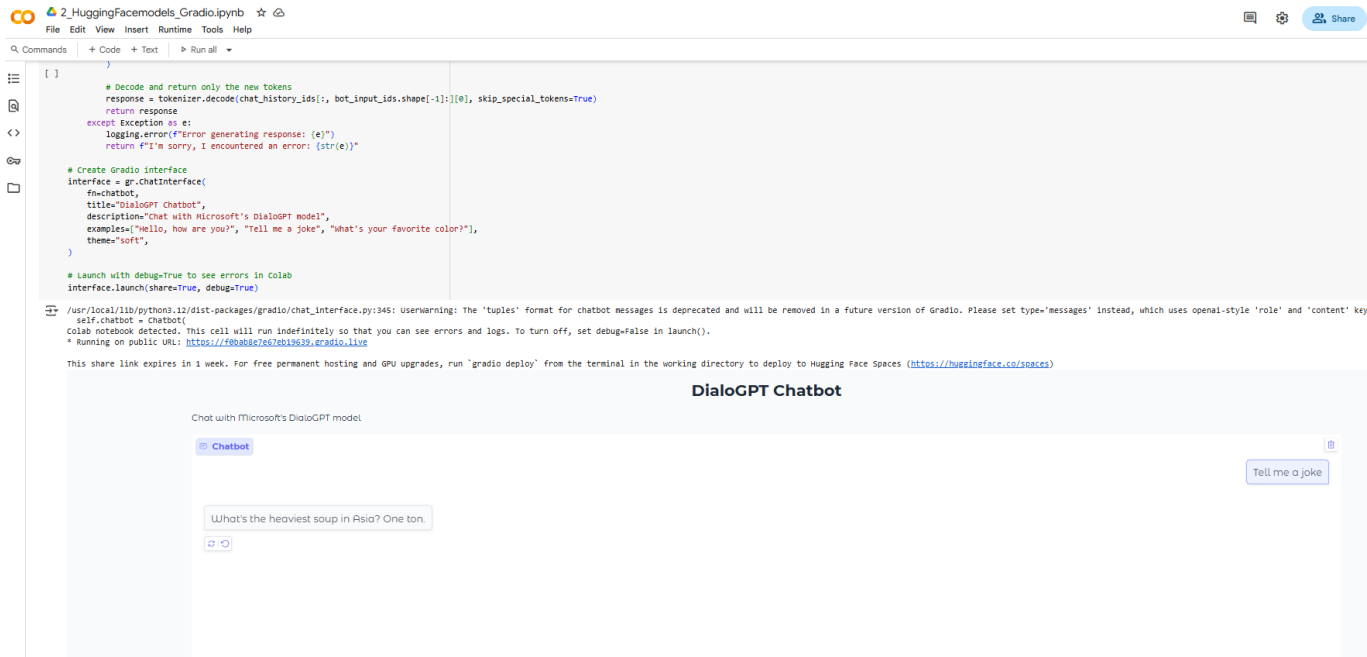
    # Generate response with attention mask
    chat_history_ids = model.generate(
        bot_input_ids,
        attention_mask=bot_attention_mask,
        max_length=1000,
        pad_token_id=tokenizer.eos_token_id,
        do_sample=True,
        temperature=0.7, # Controls randomness (higher = more random, lower = more deterministic)
        top_p=0.9,
    )

    # Decode and print response
    bot_response = tokenizer.decode(chat_history_ids[-1:], skip_special_tokens=True)
    print("Bot 🗨️: {}".format(bot_response))

Me 🗨️: hi
Bot 🗨️: Hey, how are you?
Me 🗨️: I'm good and you?
```

<https://tinyurl.com/ANLPsession5Part2>

And a Gradio
interface!



The image shows a Jupyter Notebook interface with a code editor and a Gradio chatbot interface below it.

Code Editor:

```
2_HuggingFaceModels_Gradio.ipynb
File Edit View Insert Runtime Tools Help

[ ]
# Decode and return only the new tokens
response = tokenizer.decode(chat_history_ids[-1:], skip_special_tokens=True)
return response
except Exception as e:
    logging.error(f"Error generating response: {e}")
    return f"I'm sorry, I encountered an error: {str(e)}"

# Create Gradio interface
interface = gr.ChatInterface(
    fn=chatbot,
    title="DialogPT Chatbot",
    description="Chat with Microsoft's DialogPT model",
    examples=["Hello, how are you?", "Tell me a joke", "What's your favorite color?"],
    theme="soft",
)

# Launch with debug=True to see errors in Colab
interface.launch(share=True, debug=True)
```

Gradio Chatbot Interface:

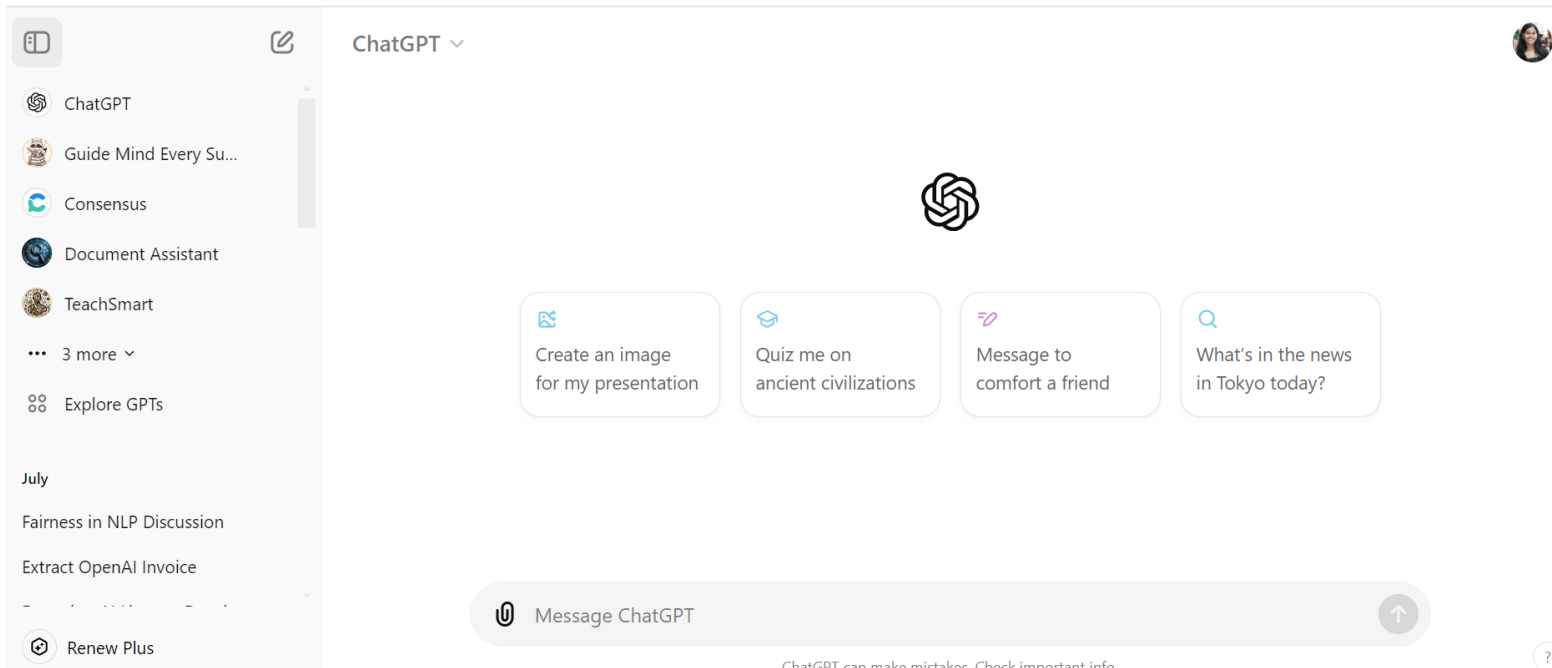
Chat with Microsoft's DialogPT model

Chatbot

What's the heaviest soup in Asia? One ton.

Tell me a joke

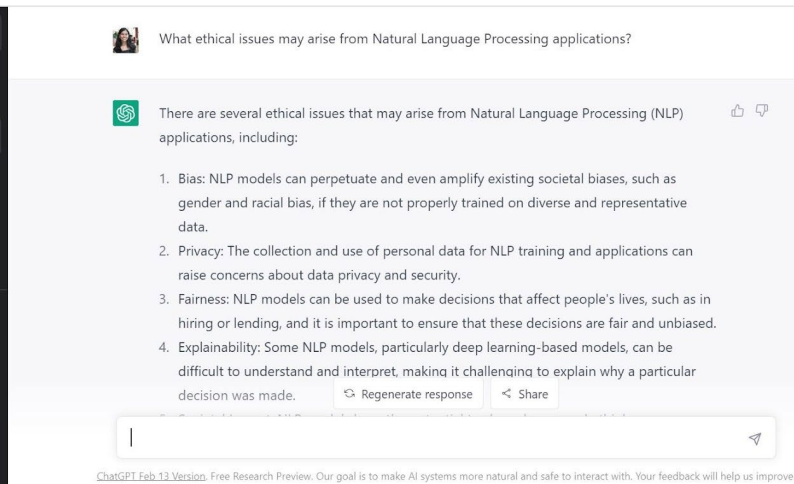
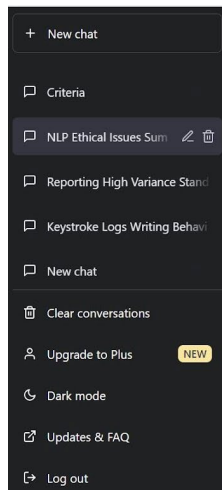
LLM based tools



Has anyone here NOT seen this screen before?

Chat GPT

- Chat GPT is an openly available chatbot developed by OpenAI that can interact with users in a conversational way.
- It was first built on GPT 3.5 and has been fine-tuned using both supervised and reinforcement learning techniques for human-like conversations (GPT-4, 4o, 5 etc. are more recently available)
- The creators use a particular technique called Reinforcement Learning from Human Feedback (RLHF), which uses human feedback in the training loop to minimize harmful, untruthful, and/or biased outputs.



Reinforcement learning (RL) – extended topic

RL is a machine learning paradigm where an "agent" learns to make decisions in an environment to maximize a reward signal, through trial and error and without explicit instructions

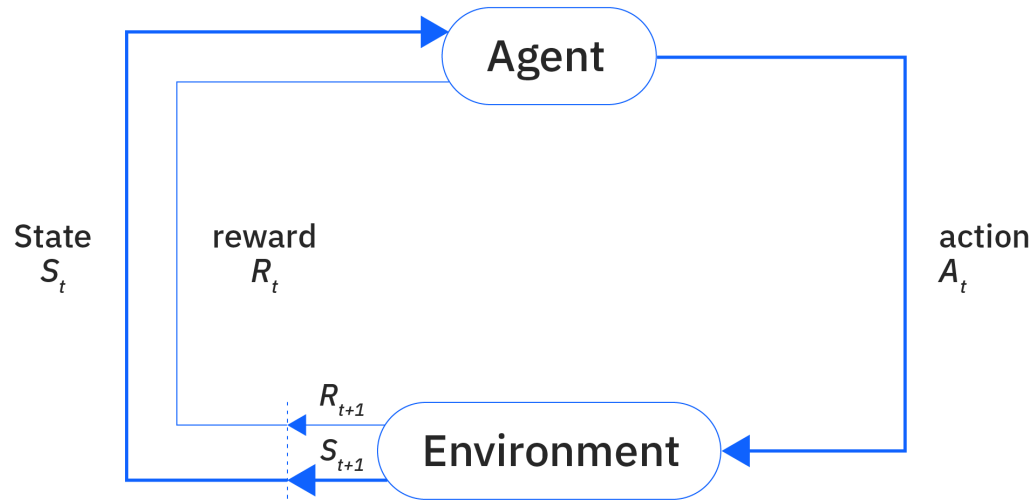


Image source: <https://www.ibm.com/think/topics/reinforcement-learning>

RLHF

<https://openai.com/index/chatgpt/>

Step 1

Collect demonstration data and train a supervised policy.

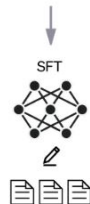
A prompt is sample from our prompt dataset.



A labeler demonstrates the desired output behavior.



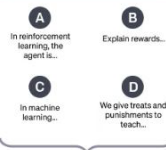
This data is used to fine-tune GPT-3.5 with supervised learning.



Step 2

Collect comparison data and train a reward model.

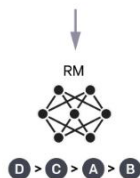
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

A new prompt is sampled from the dataset.



The PPO model is initialized from the supervised policy.



The policy generates an output.

Once upon a time...

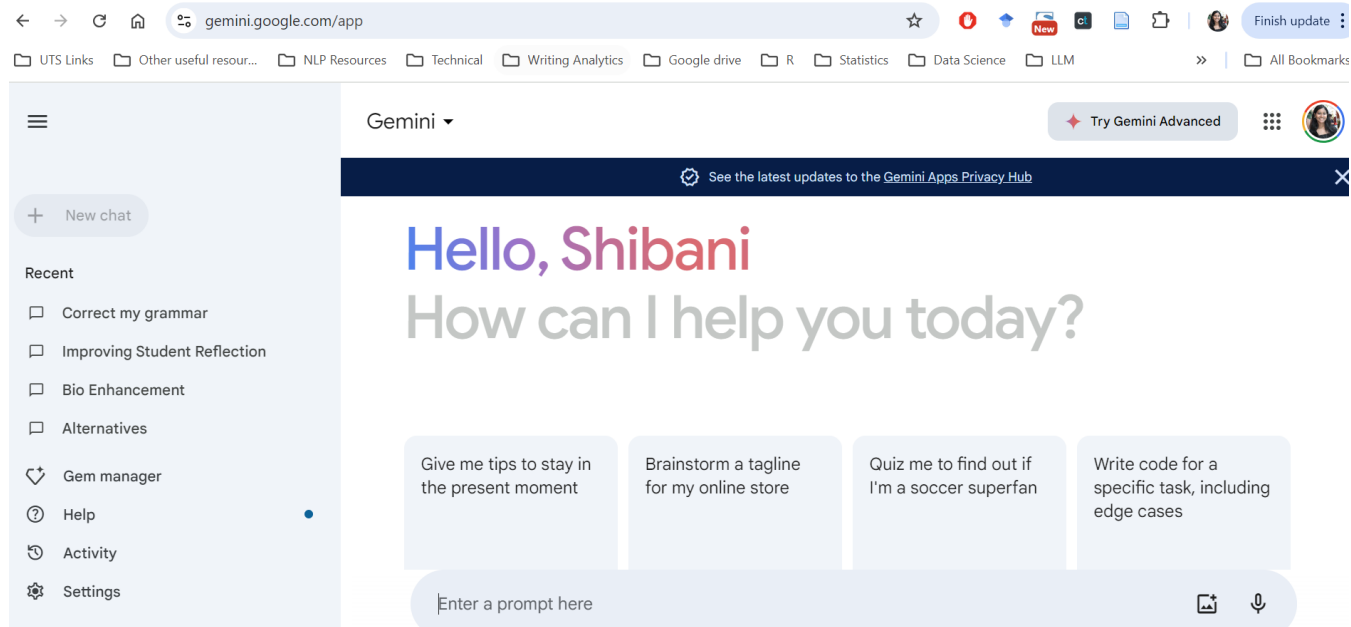
The reward model calculates a reward for the output.



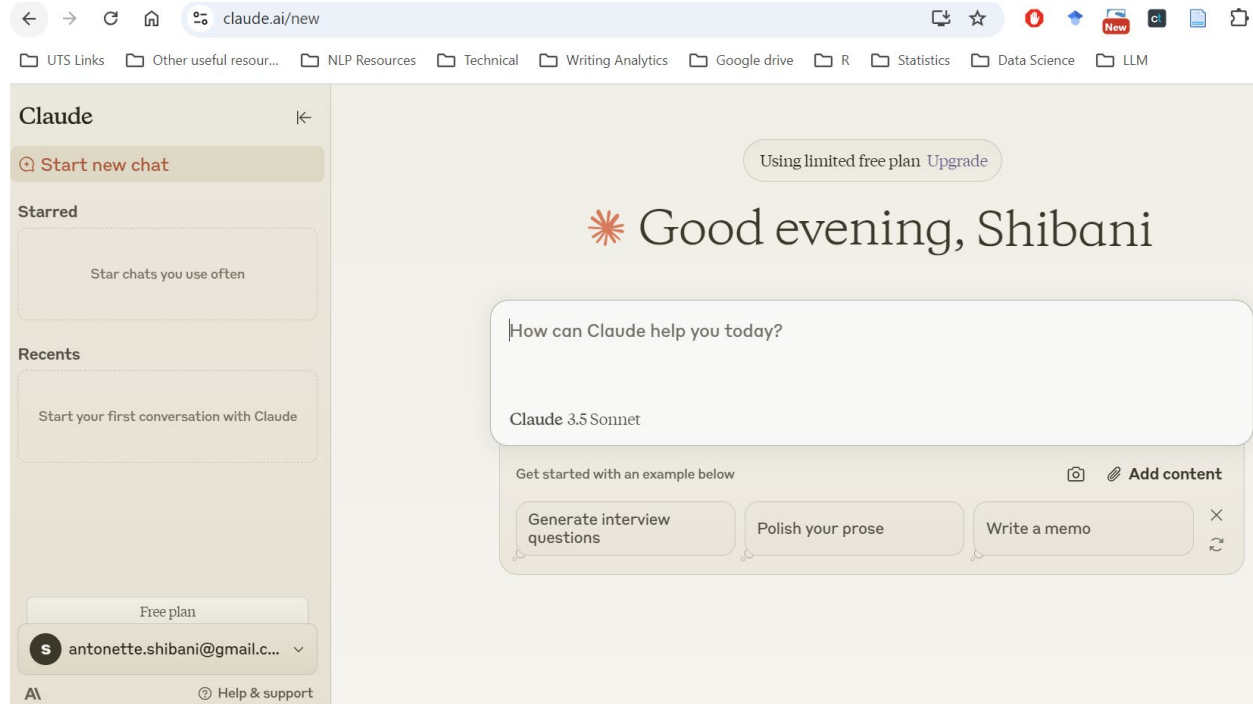
The reward is used to update the policy using PPO.

r_k

Other LLM-based chat assistants – Gemini from Google



Claude by Anthropic



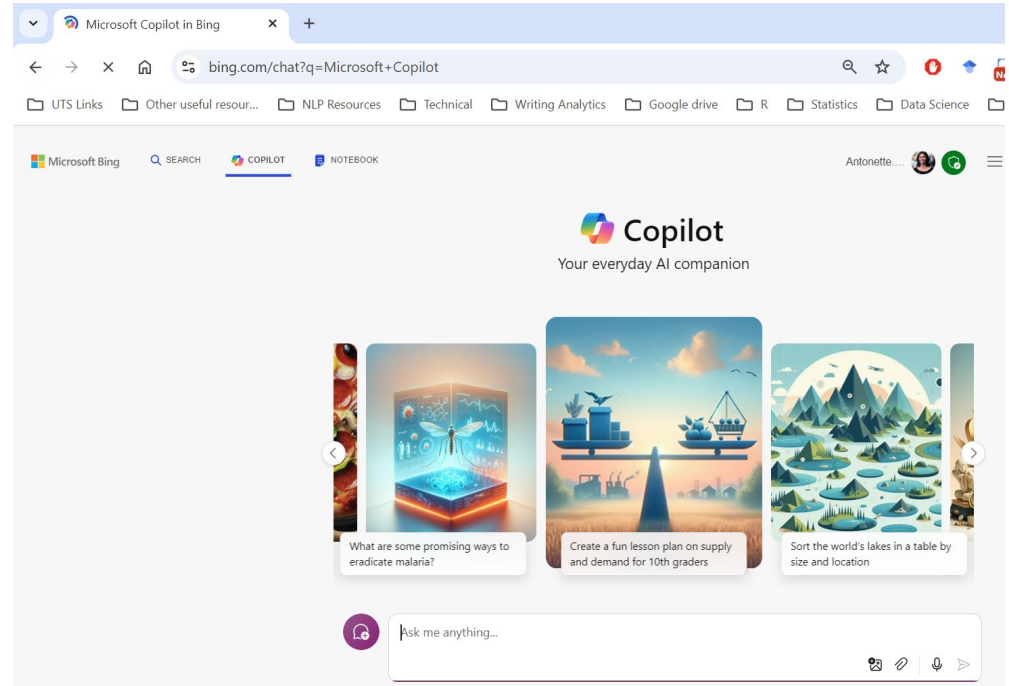
Safer,
More ethically sourced,
Works great for coding!

<https://www.anthropic.com/claude>

Bing/ Copilot by Microsoft

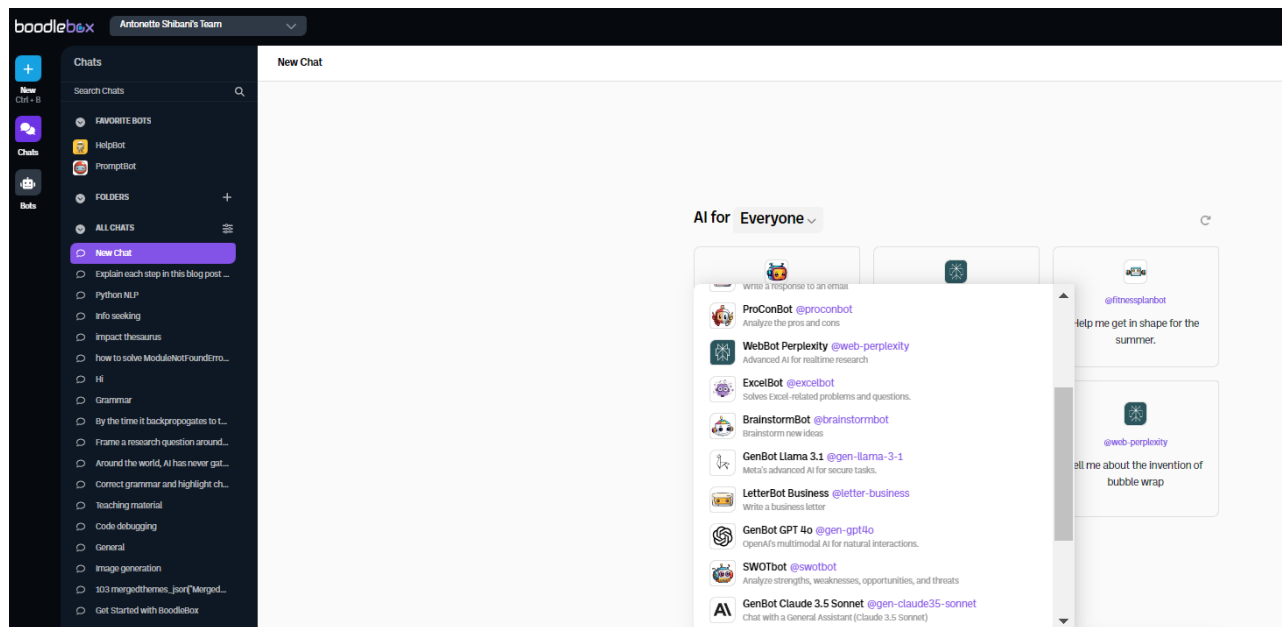
UTS has an enterprise license, the environment is **secure!**

(our chats are not used for model training)



Integrated LLM tools

E.g. Booodle box – lets you choose models by paying one subscription fee

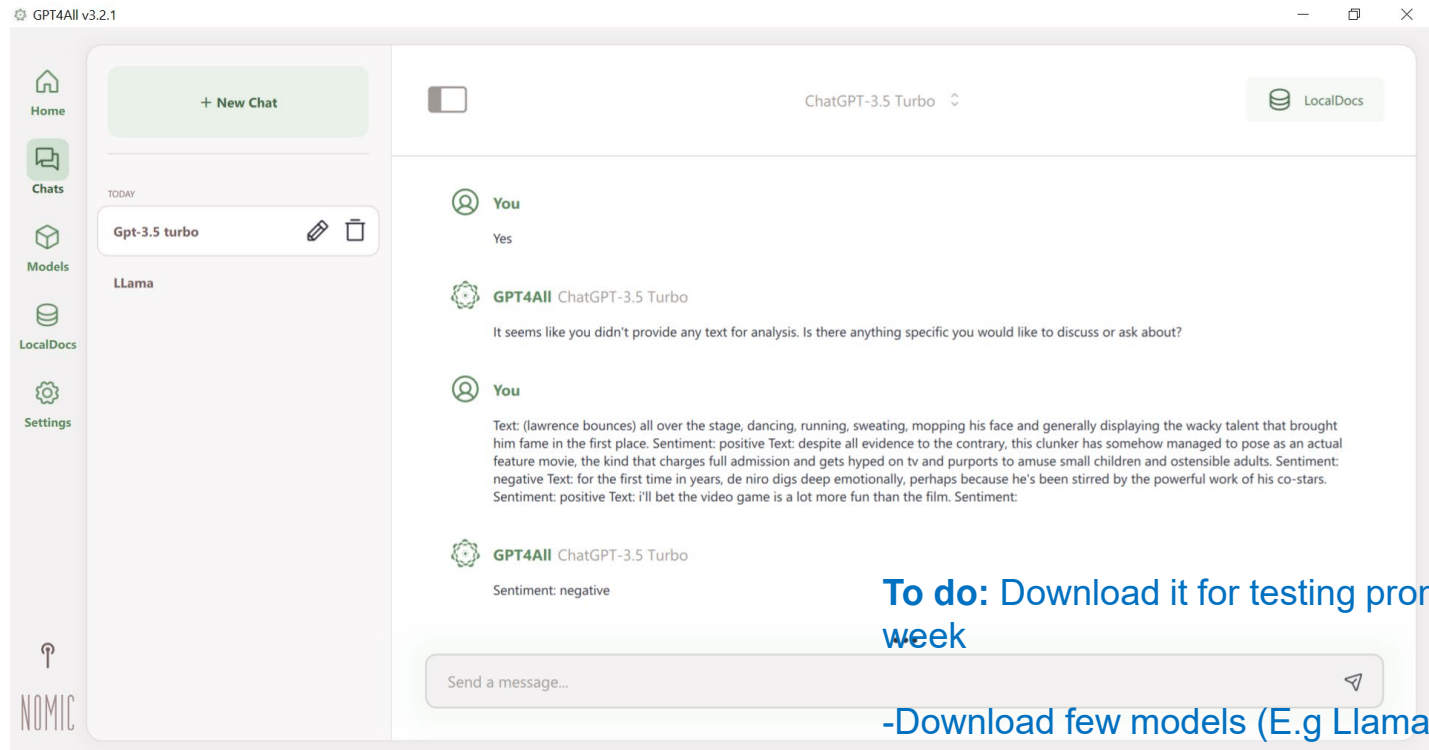


*Note: Not endorsed!

Local LLMs

E.g. GPT4All: <https://www.nomic.ai/gpt4all>

- Data sits in your local machine and computation is powered by your laptop; safe for confidential data!
- Might be slow



To do: Download it for testing prompting next week

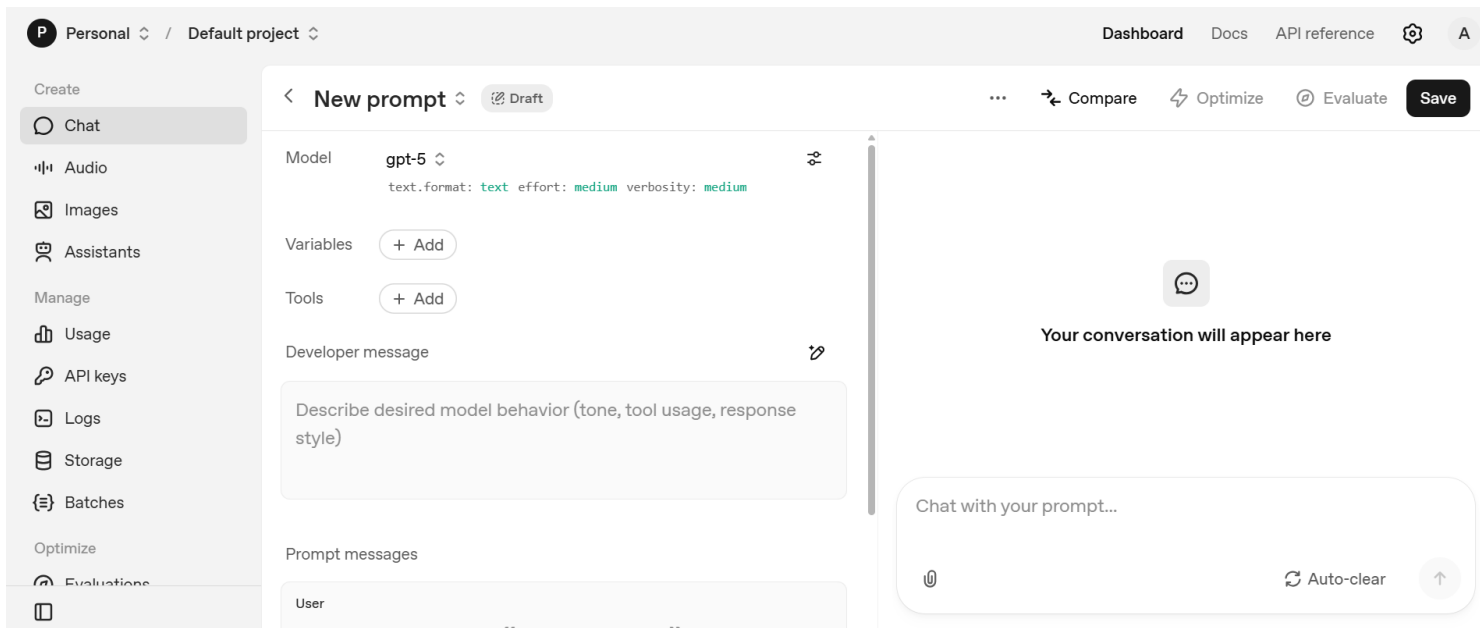
- Download few models (E.g Llama, gpt 3.5)
- Try connecting to a source data for RAG

Homework activity

- Try out the various exercises we've tried so far (e.g. NLP analysis, sentiment classification, NER etc.) using a LLM prompt. You can use any of your preferred tools - great if you can compare multiple!
- If you are a bit more advanced, go on to enable your own set up (check the following slides)

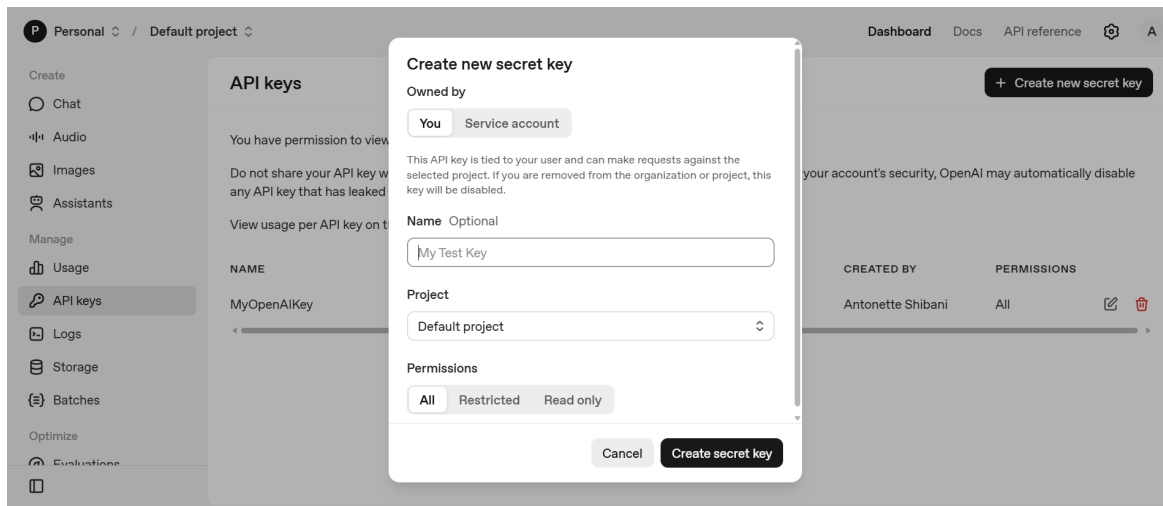
Accessing LLMs on the web

- You can try prompting LLMs on the [OpenAI Playground](#) on the web; simply **sign up for an OpenAI account using your email**



Accessing LLMs through APIs

- You will be able to create a new secret key from your OpenAI account, which will be your API key (you can use it for later in your code, etc.)
- An alternate option is accessing GPTs through Github models: <https://github.com/marketplace?type=models> or MSAzure: <https://azure.microsoft.com/en-au/products/ai-foundry/models/>



Tutorial Homework - **Setting up a local LLM-based tool**

You can use a local LLM, which is more secure and has some free options. There are many of these on the web, but we'll use one popular tool called **gpt4all**.

Download the right version for your computer to install a local copy:

<https://www.nomic.ai/gpt4all>

Get GPT4All installed in your laptop to experiment prompting strategies using local LLMs in the next session!

AT2 Group Updates

15 groups

Team Name: Group 1 Project Title: Humour analysis on Reddit for social media marketing Team member names: 1. Aarzo Katyal 2. Kinshuk Thakur 3. Amaan 4. Jaideep Confirmed: Yes Project approved	Team Name: Group 2 Project Title: NLP-based Medical Report Summariser Team member names: 1. Alison Barbosa Guzman 2. Goh Queenie 3. Li Yan 4. Ratticha Ratanawarocho 5. Tan Fang Yee Confirmed: Yes Project approved Refer to the Group Project Idea on the right for the high level information Our team will drop an email to share more on the details of our project soon! Thanks!!!	Team Name: Group 3 Project Title: Team member names: 1. Gourika Sood 2. Yash Kapasi 3. Gregory Joseph 4. Yoon Huh 5. Anupam Tiwari Confirmed: Yes	Team Name: Group 4 Project Title: Team member names: 1. Abhishek Teotia 2. Muskan Choudhary 3. Justin Govanxa 4. Yuzhong Wang Confirmed: Yes
Team Name: Group 5 Project Title: Team member names: 1. Usam Adhikari 2. Supriya Thapa 3. Aastha Dangol 4. Sophia Schwalb Confirmed: Yes Group 5: Project 1 Conversational Learning Assistant Objective & Scope: - Build a hybrid ML/DL based conversational assistant that can handle open-domain questions and provide relevant, context-aware responses. - Support personalization based on user history or pre-defined user profiles. Group 5: Project 2 Shutterbug! Mental Health Journal Companion Objective & Scope: - Build a mobile app (promoting tool) where users can track daily moods, feelings, and thoughts. - Analyze user feedback, moods, and mood trends, and suggest relevant suggestions. - Provide mental health insights without requiring professional help.	Team Name: Group 6 Project Title: Team member names: 1. Lin Li 2. Ricki Yang 3. Guorui Gao 4. Haoyu Wang 5. Weiran Sun Confirmed: Yes	Team Name: Group 7 Project Title: Team member names: 1. Swathy Mimisal Subramanian 2. Vinoth Kumar Rajagopalan 3. Summaya Maqdoom Makkiya 4. Vandana Rajagopal 5. Mitesh Radhakrishnan Confirmed: Yes	Team Name: Group 8 Project Title: Lecture Notes AI Tutor For Education Team member names: 1. Qimatx Nguyen 2. Trinh Nhat Quang Nguyen 3. Linh Chi Tong 4. Huy Duc Vu 5. Mai Loan Nguyen Confirmed: Yes Project approved

Team Name: Group 9 Project Title: Team member names: 1. Yifei He 2. Yuyang Han 3. Tianyu Wang 4. Xiaofei Jia Confirmed: Yes	Team Name: Group 10 Project Title: Team member names: 1. Alistiar Schillert 2. Brett McKeekhabn 3. Mario Rubiano 4. Shivam Jadhav Confirmed: Yes/ No Note: Full group allocated by the teaching team	Team Name: Group 11 Project Title: Team member names: 1. Yagna Sri Pailla 2. Vishal Kandikattu 3. Ketanjeet Singh 4. Ayush Sharma Confirmed: Yes/ No Note: Full group allocated by the teaching team	Team Name: Group 12 Project Title: Radiation Alerts to Action: An AI-Based Advisory Assistant for Astronaut Safety Team member names: 1. Rasyid Sahindra 2. Parth Tiwari 3. Biplav Adhikari 4. Ting Chen. Confirmed: Yes Project approved
Team Name: Group 13 Project Title: Smart To-Do List and Task Categorizer Team member names: 1. Reddy Motuku 2. Vastav Pradeep Bhagat 3. Abdulaziz Alrubayan 4. Siddhartha Reddy Gillela Confirmed: Yes Project approved	Team Name: Group 14 Project Title: Team member names: 1. MD MOHSIN HIMEL MOZUMDER 2. TAHMID HASSAN BHUIYAN 3. Nicholas Guntoro 4. Rose Jepkoech Confirmed: Yes/ No Note: Full group allocated by the teaching team	Team Name: Group 15 Project Title: Team member names: 1. Faiyaz Bin Khaled 2. Saurav Parajuli 3. Weishu Sun 4. Pai Ye Confirmed: Yes/ No Note: Full group allocated by the teaching team	